

《算法设计与分析》

期末突击课



考点	重要程度	占分	题型
1.算法的特性	★★★★★	2-4	填空、选择、判断、简答
2.算法的时间和空间复杂度	★★★★	2-4	填空、选择、简答
3.算法的渐进表示	★★★★	2-5	填空、简答

考点1 算法的特性（掌握）

算法：解决问题的一种方法或一个过程。

算法的例子：辗转相除法、刘徽割圆术、冒泡排序等。

算法的五个特性：有穷性、确定性、可行性、输入、输出。

“好”算法的特质：正确性、可读性、健壮性、高效率与低存储量需求。

算法与程序的区别：程序 = 数据结构 + 算法。



扫码观看视频讲解更清晰

考点2 算法的时间和空间复杂度（掌握）

算法复杂度分析是指对一个算法所需的计算资源进行预测。最重要的计算资源是时间和空间资源。因此，一个算法的优劣可以用时间复杂度与空间复杂度来衡量。

时间复杂度：是指对于特定输入该算法对该输入产生结果所需要的原子操作或步数，时间开销 $T(n)$ 是问题规模即输入大小 n 的函数。

最坏时间复杂度：最坏情况下算法的时间复杂度。

平均时间复杂度：所有输入示例等概率出现的情况下，算法的期望运行时间。

最好时间复杂度：最好情况下的时间复杂度。

空间复杂度：是指对于特定输入该算法对该输入产生结果所需要的存储空间的大小。

【题1】 冒泡排序执行过程中每一条语句的执行时间开销和执行次数如下所示，分别计算冒泡排序的时间开销、最好情况下时间开销和最坏情况下时间开销。设 $n=length[A]$ ， t_i 为第四行执行的次数，注释部分是不可执行的语句，不占运行时间。

BUBBLE-SORT(A)	开销	次数
1 for $i \leftarrow 1$ to $length[A]$	c_1	$n+1$
2 do for $j \leftarrow length[A]$ downto $i+1$	c_1	$\sum_{i=1}^n (n-i+1)$
3 do if $A[j] < A[j-1]$	c_2	$\sum_{i=1}^n (n-i)$
4 then exchange $A[j] \leftrightarrow A[j-1]$	c_3	t_i

答：该算法的总运行时间是每一条语句执行时间之和，由此可得算法的运行时间 $T(n)$ 为：

$$\begin{aligned}
 T(n) &= c_1(n+1) + c_2\left[\sum_{i=1}^n (n-i+1)\right] + c_2\left[\sum_{i=1}^n (n-i)\right] + c_3t_i \\
 &= c_1(n+1) + c_1 \frac{n(n+1)}{2} + c_2 \frac{n(n-1)}{2} + c_3t_i \\
 &= \frac{c_1+c_2}{2}n^2 + \frac{3c_1-c_2}{2}n + c_1 + c_3t_i
 \end{aligned}$$



扫码观看视频讲解更清晰

【题1】 冒泡排序执行过程中每一条语句的执行时间开销和执行次数如下所示，分别计算冒泡排序的时间开销、最好情况下时间开销和最坏情况下时间开销。设 $n=length[A]$ ， t_i 为第四行执行的次数，注释部分是不可执行的语句，不占运行时间。

BUBBLE-SORT(A)	开销	次数
1 for $i \leftarrow 1$ to $length[A]$	c_1	$n+1$
2 do for $j \leftarrow length[A]$ downto $i+1$	c_1	$\sum_{i=1}^n (n-i+1)$
3 do if $A[j] < A[j-1]$	c_2	$\sum_{i=1}^n (n-i)$
4 then exchange $A[j] \leftrightarrow A[j-1]$	c_3	t_i

当输入数组正好为升序时，出现最好情况，第4句语句不执行，即 $t_i=0$ ，此时运行时间为：

$$T(n) = \frac{c_1 + c_2}{2} n^2 + \frac{3c_1 - c_2}{2} n + c_1$$

当输入数组正好为降序时，出现最坏情况，第4句语句执行次数为 $\sum_{i=1}^n (n-i)$ ，即 $t_i=n(n-1)/2$ ，此时运行时间为：

$$T(n) = \frac{c_1 + c_2 + c_3}{2} n^2 + \frac{3c_1 - c_2 - c_3}{2} n + c_1$$

考点3 算法的渐近表示（理解）

一、复杂度函数的阶的计算

如何计算复杂度函数的阶？保留最高阶项，忽略低阶项，忽略常数项。

常见的阶有： $\Theta(1)$, $\Theta(\log n)$, $\Theta(\sqrt{n})$, $\Theta(n)$, $\Theta(n \log n)$, $\Theta(n^2)$, $\Theta(n^3)$, $\Theta(2^n)$, $\Theta(n!)$ 。

二、渐进表示

渐近表示符号： Θ 、 O 、 Ω 。

1. 如果存在三个正常数 c_1 , c_2 , n_0 , 对于所有的 $n \geq n_0$, 有 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$, 则记作 $f(n) = \Theta(g(n))$, 并且称 $g(n)$ 是 $f(n)$ 的一个渐近紧致界, 也就是说 $g(n)$ 与 $f(n)$ 同阶。

【题2】 证明： $\frac{1}{2}n^2 - 5n = \Theta(n^2)$

证： 根据定义，要确定常数 c_1, c_2, n_0 ，使得对于所有 $n \geq n_0$ ，有

$$c_1 n^2 \leq \frac{1}{2}n^2 - 5n \leq c_2 n^2$$

化简可得

$$c_1 \leq \frac{1}{2} - \frac{5}{n} \leq c_2$$

左边的不等式在 $n \geq 11, c_1 \leq 1/22$ 时成立；右边的不等式在 $n \geq 1, c_2 \geq 1/2$ 时成立。

这样，可以选择 $0 < c_1 \leq 1/22, c_2 \geq 1/2$ 以及 $n_0 = 11$ ，从而就证明了 $\frac{1}{2}n^2 - 5n = \Theta(n^2)$ 。

注意： 这里常数的选择不是唯一的。

考点3 算法的渐近表示（理解）

二、渐进表示

2. 如果存在两个正常数 c, n_0 , 对于所有的 $n \geq n_0$, 有 $0 \leq f(n) \leq cg(n)$, 则记作 $f(n) = O(g(n))$, 并且称 $g(n)$ 是 $f(n)$ 的一个上界函数, 也就是说 $f(n)$ 是 $g(n)$ 的低阶函数。



扫码观看视频讲解更清晰

【题3】 证明： $\frac{1}{3}n^2 - 3n = O(n^2)$

证： 根据定义，要确定常数 c ， n_0 ，使得对于所有 $n \geq n_0$ ，有

$$\frac{1}{3}n^2 - 3n \leq cn^2$$

化简可得

$$\frac{1}{3} - \frac{3}{n} \leq c$$

不等式在 $n \geq 1$ ， $c \geq 1/3$ 时成立。这样，可以选择 $c \geq 1/3$ 以及 $n_0 = 1$ ，

从而就证明了 $\frac{1}{3}n^2 - 3n = O(n^2)$ 。

考点3 算法的渐近表示（理解）

二、渐进表示

3. 如果存在两个正常数 c, n_0 , 对于所有的 $n \geq n_0$, 有 $0 \leq cg(n) \leq f(n)$, 则记作 $f(n) = \Omega(g(n))$, 并且称 $g(n)$ 是 $f(n)$ 的一个下界函数, 也就是说 $f(n)$ 是 $g(n)$ 的高阶函数。



扫码观看视频讲解更清晰

【题4】 证明： $\frac{1}{3}n^2 - 3n = \Omega(n^2)$

证：根据定义，要确定常数 c ， n_0 ，使得对于所有 $n \geq n_0$ ，有

$$cn^2 \leq \frac{1}{3}n^2 - 3n$$

化简可得

$$c \leq \frac{1}{3} - \frac{3}{n}$$

不等式在 $n \geq 18$ ， $0 < c \leq 1/6$ 时成立。这样，可以选择 $0 < c \leq 1/6$ 以及 $n_0=18$ ，从而就证明了 $\frac{1}{3}n^2 - 3n = \Omega(n^2)$ 。

考点3 算法的渐近表示（理解）

三、常见时间复杂度比较（常对幂指阶）

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) \\ < O(n!) < O(n^n)$$

【题5】 写出下列复杂度函数的排序关系（即按照渐近阶从低到高排序）。

$$2^n \quad 3^n \quad \log n \quad n! \quad n \log n \quad n^2 \quad n^n \quad 10^3$$

解： $10^3 < \log n < n \log n < n^2 < 2^n < 3^n < n! < n^n$



《算法设计与分析》

期末突击课

考点	重要程度	占分	题型
1.递归与递归方程	★★★★	2-5	填空、判断、计算
2.分治法的基本思想	★★★★	2-5	填空、选择、简答
3.分治法应用实例	★★★★★	2-8	填空、选择、简答、算法设计



扫码观看视频讲解更清晰

考点1 递归与递归方程（掌握）

一、递归的概念

递归：程序对自己的调用。

递归需要满足的三个条件：

1. 一个问题的解可以分解为几个子问题的解；
2. 这个问题与分解之后的子问题，除了数据规模不同，求解思路完全一样；
3. 存在递归终止条件。

关于递归的例子：斐波那契序列、欧几里得算法、汉诺塔问题、阶乘等。

考点1 递归与递归方程（掌握）

二、递归方程的建立

当算法包含调用自身的过程时，其运行时间可用递归方程描述。

下面是递归方程建立的具体过程：假设问题规模为 n ， $T(n)$ 为解决该问题的时间开销。

- 1、当 $n < c$ 时， $T(n) = \Theta(1)$ 。
- 2、当 $n \geq c$ 时，
 - (1) 划分阶段的时间开销：划分问题为 a 个子问题，每个子问题规模为 n/b ，划分时间 $D(n)$ ；
 - (2) 递归求解阶段的时间开销：递归调用，求解时间 $aT(n/b)$ ；
 - (3) 合并阶段的时间开销：合并时间 $C(n)$ 。

因此，可得递归方程
$$T(n) = \begin{cases} \Theta(1), n < c \\ aT(n/b) + D(n) + C(n), n \geq c \end{cases}$$

考点1 递归与递归方程（掌握）

三、递归方程的求解

常用的求解递归方程的方法有两种：**替换方法和主定理**。

1. 替换方法

用替换方法解某个递归方程时，分为两步。

首先是猜测问题解的某个界限，然后用数学归纳法证明所猜测解的正确性。

猜测问题的界限可以根据经验猜，也可以把递归方程逐项展开，再对项进行合并，根据合并结果猜测问题的界限。

【题1】解递归方程 $T(n) = 2T(n/2) + n$ 。

解：猜测其解为 $T(n) = O(n \log_2 n)$

假设 n 为 2 的幂且这个界限对于 $n/2$ 成立，即存在某个常数 c ，使得

$T(n/2) \leq c(n/2) \log_2(n/2)$ 成立。现在要证明 $T(n) \leq cn \log_2 n$ 。将假设带入递归

方程可得 $T(n) = 2T(n/2) + n$

$$\leq 2(c(n/2) \log_2(n/2)) + n$$

$$= cn \log_2(n/2) + n$$

$$= cn \log_2 n - cn \log_2 2 + n$$

$$= cn \log_2 n - cn + n$$

$$= cn \log_2 n - (c-1)n \leq cn \log_2 n$$

最后一步在 $c \geq 1$ 时成立。



扫码观看视频讲解更清晰

【题1】 解递归方程 $T(n)=2T(n/2)+n$ 。

接下来猜测对于边界条件也成立，即证明对于选择的常数 c ， $T(n) \leq cn \log_2 n$ 对于边界条件成立。

假设 $T(1)=1$ 是递归方程的边界条件，那么对于 $n=1$ ， $T(1) \leq c \cdot 1 \cdot \log_2 1 = 0$ 与 $T(1)=1$ 矛盾。

设 $n_0=2$ ，选择 $T(2)$ 作为归纳证明中的边界条件。由递归方程可得 $T(2)=4$ 。

此时只要选择 $c \geq 2$ ，就会使得 $T(2) \leq c \cdot 2 \cdot \log_2 2$ 成立。

因此，只要选择 $n_0=2$ 和 $c \geq 2$ ，就有 $T(n) \leq cn \log_2 n$ 成立，

也即 $T(n) = O(n \log_2 n)$ 。

【题2】 解递归方程 $T(n)=3T(n/4)+n^2$ 。

解： 假设 n 为4的幂，把递归方程 $T(n)$ 逐项展开，可得

$$\begin{aligned}T(n) &= 3T(n/4) + n^2 \\&= 3(3T(n/4^2) + (n/4)^2) + n^2 \\&= 3^2T(n/4^2) + 3(n/4)^2 + n^2 \\&= 3^2T(n/4^2) + (3/16)n^2 + n^2 \\&= 3^2(3T(n/4^3) + (n/4^2)^2) + (3/16)n^2 + n^2 \\&= 3^3T(n/4^3) + 3^2(n/4^2)^2 + (3/16)n^2 + n^2 \\&= 3^3T(n/4^3) + (3/16)^2n^2 + (3/16)n^2 + n^2 \\&\dots\dots\end{aligned}$$

$$\begin{aligned}&= 3^{\log_4 n}T(1) + (3/16)^{\log_4 n-1}n^2 + \dots + (3/16)^2n^2 + (3/16)n^2 + n^2 \\&= 3^{\log_4 n}T(1) + \sum_{i=0}^{\log_4 n-1} (3/16)^i n^2 \\&\leq 3^{\log_4 n}T(1) + \sum_{i=0}^{\infty} (3/16)^i n^2 \\&= n^{\log_4 3}T(1) + \frac{1}{1-3/16}n^2 \\&= \Theta(n^{\log_4 3}) + (16/13)n^2 \\&= O(n^2)\end{aligned}$$

【题2】 解递归方程 $T(n)=3T(n/4)+n^2$ 。

在这里，猜测 $T(n)=O(n^2)$ 。接下来用替换方法证明猜测的正确性。

假设这个界限对于 $n/4$ 成立，即存在正常数 c ， $T(n/4) \leq c(n/4)^2$ 成立。

代入递归方程可得

$$\begin{aligned} T(n) &= 3T(n/4) + n^2 \\ &\leq 3c(n/4)^2 + n^2 \\ &= (3/16)cn^2 + n^2 \\ &\leq cn^2 \end{aligned}$$

因此，选取 $c \geq (16/13)$ ，最后一步便成立，也即 $T(n)=O(n^2)$ 得证。

【题3】 解递归方程 $T(n)=2T(\sqrt{n})+1$

解： 设 $n=2^m$ 或者 $m=\log_2 n$ ， 则递归方程变为

$$T(2^m)=2T(2^{m/2})+1$$

再做一次替换， 设 $S(m)=T(2^m)$ ， 则递归方程变为

$$S(m)=2S(m/2)+1$$

该方程的解为 $S(m)=\Theta(m)$ 。 换成T， 可得 $T(2^m)=\Theta(m)$ 。

将 $n=2^m$ 带入， 得到

$$T(n)=\Theta(\log_2 n)$$

考点1 递归与递归方程（掌握）

2. 主定理

设 $a \geq 1$, $b > 1$ 为常数, $f(n)$ 为一函数。 $T(n)$ 由以下递归方程定义:

$$T(n) = aT(n/b) + f(n)$$

其中 n 为非负整数, $T(n)$ 则有如下的渐近界限:

- (1) 若对某些常数 $\varepsilon > 0$, 有 $f(n) = O(n^{\log_b a - \varepsilon})$, 那么 $T(n) = \Theta(n^{\log_b a})$ 。
- (2) 若 $f(n) = \Theta(n^{\log_b a})$, 那么 $T(n) = \Theta(n^{\log_b a} \log n)$ 。
- (3) 若对某些常数 $\varepsilon > 0$, 有 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, 且对常数 $c < 1$ 与所有足够大的 n , 有 $af(n/b) \leq cf(n)$, 那么 $T(n) = \Theta(f(n))$ 。

考点1 递归与递归方程（掌握）

在上述每一种情况下，都把函数 $f(n)$ 与函数 $n^{\log_b a}$ 进行比较，递归方程的解由这两个函数中较大的一个决定。

在第一种情形中，函数 $n^{\log_b a}$ 比函数 $f(n)$ 更大，则递归方程解为

$$T(n) = \Theta(n^{\log_b a})$$

在第二种情形中，这两个函数一样大也即同阶，则递归方程解为

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(f(n) \log n)$$

在第三种情形中，函数 $f(n)$ 比函数 $n^{\log_b a}$ 大，则递归方程解为

$$T(n) = \Theta(f(n))$$

考点1 递归与递归方程（掌握）

在第一种情形中，函数 $f(n)$ 不仅比函数 $n^{\log_b a}$ 要小，还必须是多项式地小于，即对于某些常数 $\varepsilon > 0$ ， $f(n)$ 渐近地小于 $n^{\log_b a}$ ，两者差一个因子 n^ε 。

在第三种情形中，函数 $f(n)$ 不仅比函数 $n^{\log_b a}$ 要大，还必须是多项式地大于，此外还要满足条件 $af(n/b) \leq cf(n)$ 。



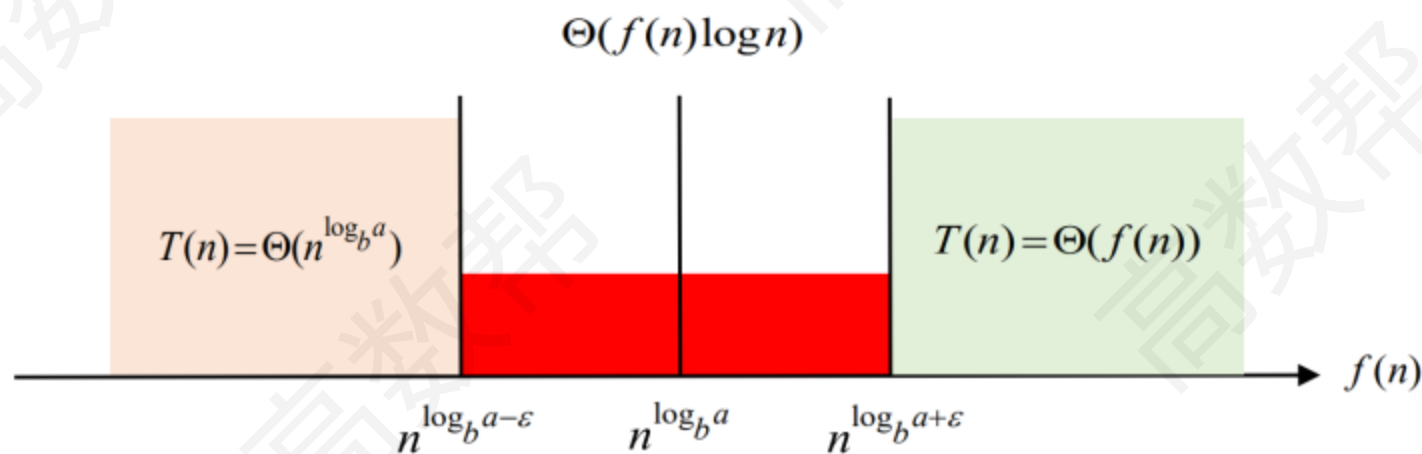
扫码观看视频讲解更清晰

考点1 递归与递归方程（掌握）

需要注意的是，这三种情形并没有覆盖所有可能的 $f(n)$ 。

如下图所示，当 $f(n)$ 不是多项式地小于 $n^{\log_b a}$ 时，在第一种情形与第二种情形之间就会出现一条间隙。

当 $f(n)$ 不是多项式地大于 $n^{\log_b a}$ 时，在第二种情形与第三种情形之间就会出现一条间隙。如果函数 $f(n)$ 落入任何一个间隙中，或者第三种情形中的条件 $af(n/b) \leq cf(n)$ 不成立，那么就不能用主定理求解递归方程。



【题4】 解递归方程 $T(n)=4T(n/2)+n$ 。

解： 由递归方程可得 $a=4$, $b=2$ 且 $f(n)=n$ 。因此, $n^{\log_b a - \varepsilon} = n^{\log_2 4 - \varepsilon} = n^{2 - \varepsilon}$ 。

选取 $0 < \varepsilon < 1$, 则

$$f(n) = O(n^{2 - \varepsilon}) = O(n^{\log_b a - \varepsilon})$$

递归方程满足主定理的第一种情形, 因此

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 4}) = \Theta(n^2)$$

【题5】 解递归方程 $T(n)=4T(n/2)+n^2$ 。

解：由递归方程可得 $a=4$, $b=2$ 且 $f(n)=n^2$ 。因此, $n^{\log_b a} = n^{\log_2 4} = n^2$ 。

$$\text{则 } f(n) = \Theta(n^2) = \Theta(n^{\log_b a})$$

递归方程满足主定理的第二种情形, 因此

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(n^{\log_2 4} \log n) = \Theta(n^2 \log n)$$



扫码观看视频讲解更清晰

【题6】解递归方程 $T(n)=4T(n/2)+n^3$ 。

解：由递归方程可得 $a=4$ ， $b=2$ 且 $f(n)=n$ 。因此， $n^{\log_b a + \varepsilon} = n^{\log_2 4 + \varepsilon} = n^{2+\varepsilon}$ 。

选取 $0 < \varepsilon < 1$ ，则 $f(n) = \Omega(n^{2+\varepsilon}) = \Omega(n^{\log_b a + \varepsilon})$

递归方程满足主定理的第三种情形，除此之外，还需证明 $af(n/b) \leq cf(n)$ 。

选择 $1/2 \leq c$ ，则 $(1/2)n^3 \leq cn^3$ 成立，即 $4(n/2)^3 \leq cn^3$ ，

也即 $4f(n/2) \leq cf(n)$ 成立。

因此，选择 $1/2 < c < 1$ ，可得 $T(n) = \Theta(f(n)) = \Theta(n^3)$

【题7】 解递归方程 $T(n) = 2T(n/2) + n\log_2 n$ 。

解： 由递归方程可得 $a=2$ ， $b=2$ 且 $f(n)=n\log_2 n$ ，那么 $n^{\log_b a} = n^{\log_2 2} = n$ 。

虽然 $f(n)=n\log_2 n$ 大于 n ，但并不是多项式地比 n 大，因此不能用主定理的第三种情况解这个递归方程。要想求解此递归方程可用替换法。



扫码观看视频讲解更清晰

考点2 分治法的基本思想（理解）

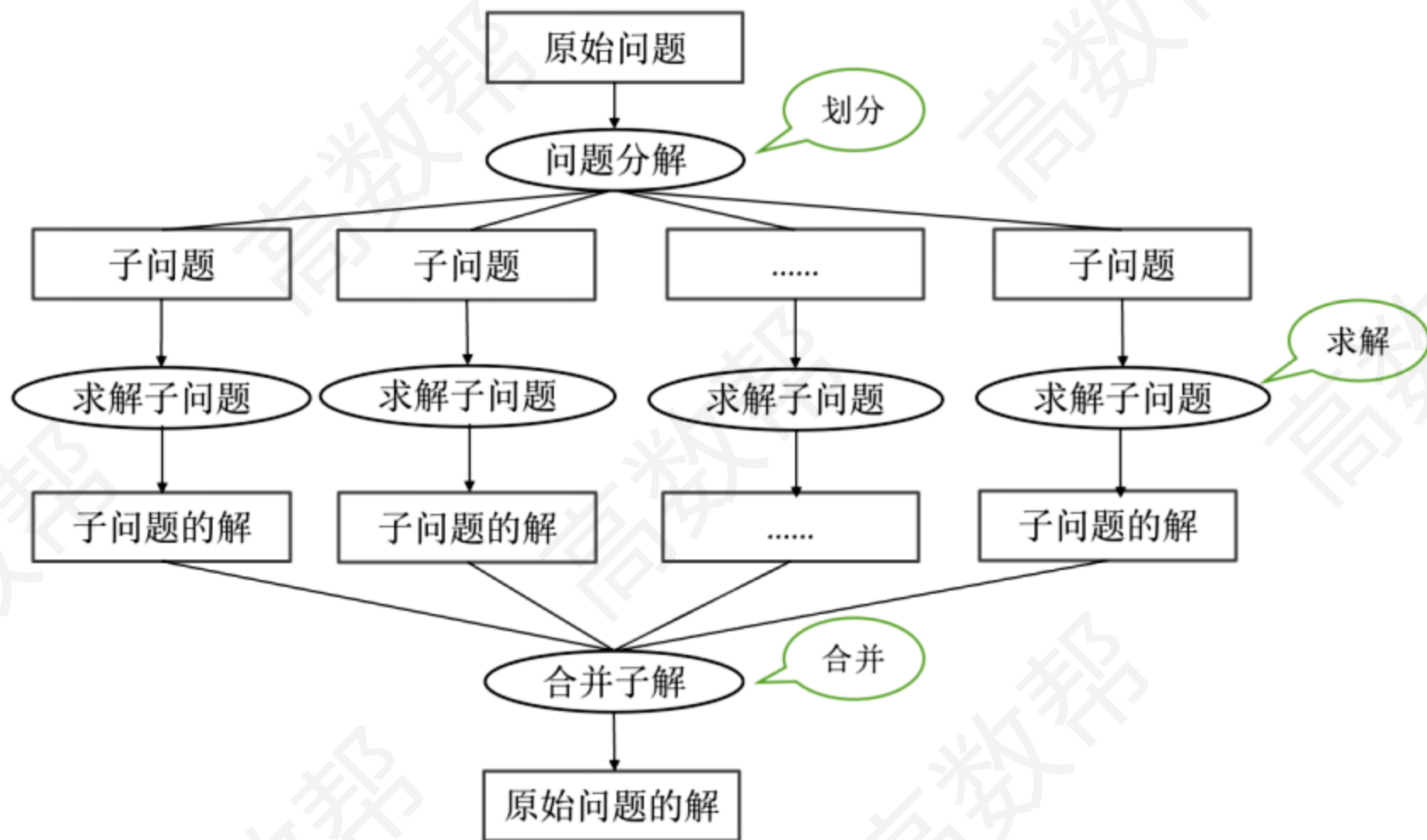
一、分治法的基本思想

对于一个规模为 n 的问题，若该问题可以容易地解决(比如说规模 n 较小)，则直接解决，否则将其分解为 k 个规模较小的子问题，这些子问题互相独立且与原问题形式相同，递归地解这些子问题，然后将各子问题的解合并，得到原问题的解。

应用分治法的两个前提是问题的可分解性和解的可归并性。

出于“平衡子问题”的思想，通常分治法在分解原问题时，形成若干子问题，这些子问题的规模都大致相同。

考点2 分治法的基本思想（理解）



分治法图解

考点2 分治法的基本思想（理解）

二、分治法的设计与分析

分治法的设计过程分为三个阶段：

- (1)划分：将原问题分解为若干规模较小、相互独立、与原问题形式相同的子问题。
- (2)解决：若子问题规模较小，则直接求解；否则递归求解各子问题。
- (3)合并：将各子问题的解合并为原问题的解。

分治法的分析过程分为两个阶段：

- (1)建立递归方程；
- (2)求解递归方程。



扫码观看视频讲解更清晰

考点3 分治法应用实例（掌握）

一、找最大值与最小值算法（MAXMIN算法）

1. MAXMIN算法的基本思想：每次将问题分成大致相等的两部分，分别在这两部分中找出最大值与最小值，再将这两个子问题的解组合成原问题的解。

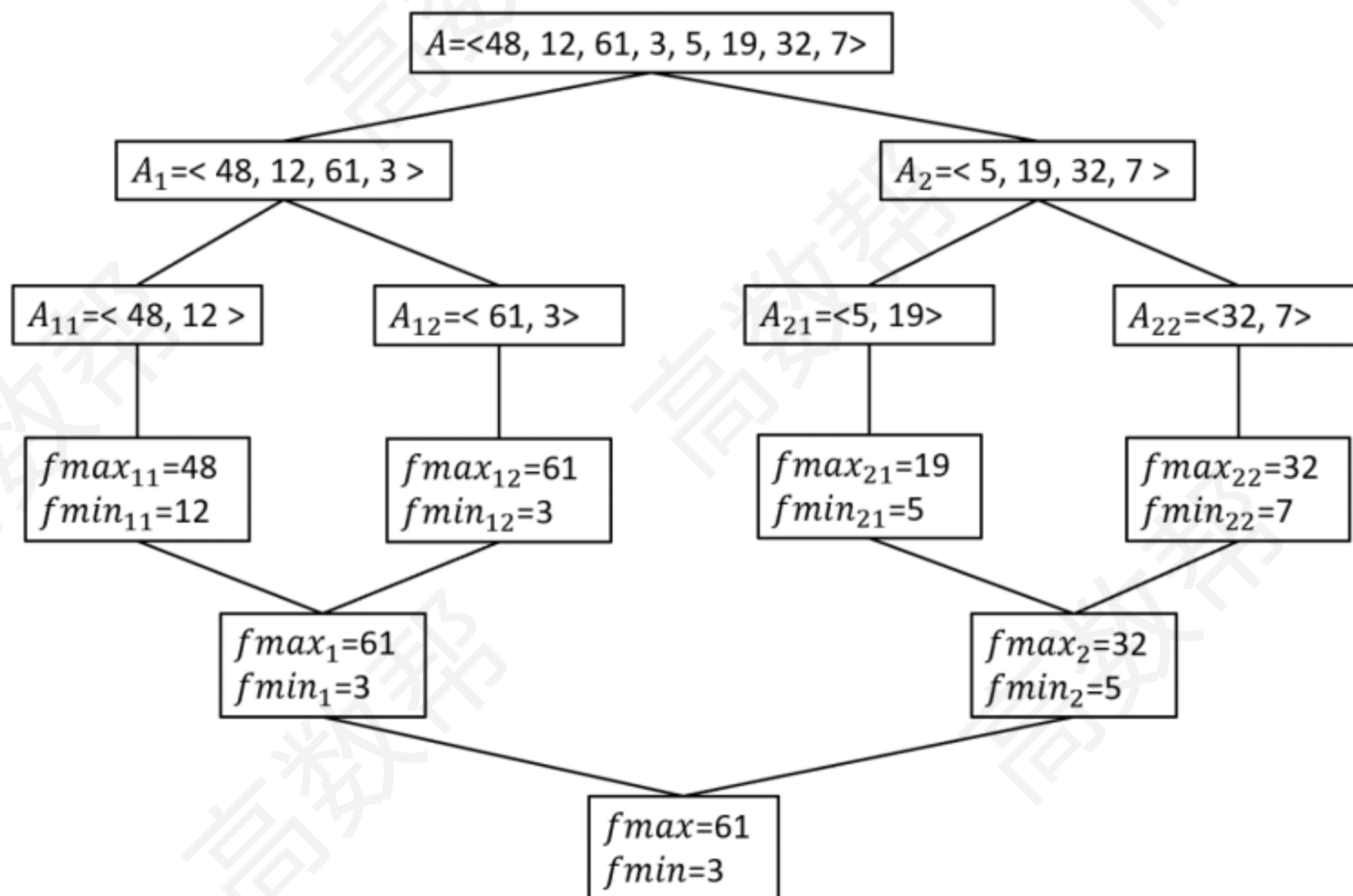
2. MAXMIN算法比较次数：

n 为偶数时，比较 $\frac{3}{2}n-2$ 次；当 n 为奇数时，比较 $3\lfloor n/2 \rfloor$ 次。

【题8】 写出MAXMIN算法对下列实例中找最大数和最小数的过程。

数组 $A=(48,12,61,3,5,19,32,7)$ 。

解：



考点3 分治法应用实例（掌握）

二、二分查找算法

1. 二分查找算法的基本思想

二分查找算法是基于分治策略实现的，

其基本思想是：对一个按照非降序排列的含有 n 个元素的数组 A 和给定元素 v ，将 n 个元素分成大致相等的两部分，取 $A([n/2])$ 与 v 进行比较，如果相等，则找到 v ，返回 v 所在位置，算法终止；

考点3 分治法应用实例（掌握）

二、二分查找算法

如果 $v < A([n/2])$ ，则在数组的左半部分继续查找 v ；

如果 $v > A([n/2])$ ，则在数组的右半部分继续查找 v 。上述过程被反复递归调用。

当所查找的区间为0时，表示 v 不在数组中，返回查找不成功标志0。

二分查找只适应于顺序存储结构。

2.二分查找算法的最坏时间复杂度： $O(\log_2 n)$



扫码观看视频讲解更清晰

考点3 分治法应用实例（掌握）

三、归并排序算法

1. 归并排序的基本思想

归并排序(merge sorting)是分治法应用的一个实例，基于分治思想，由以下三步组成：

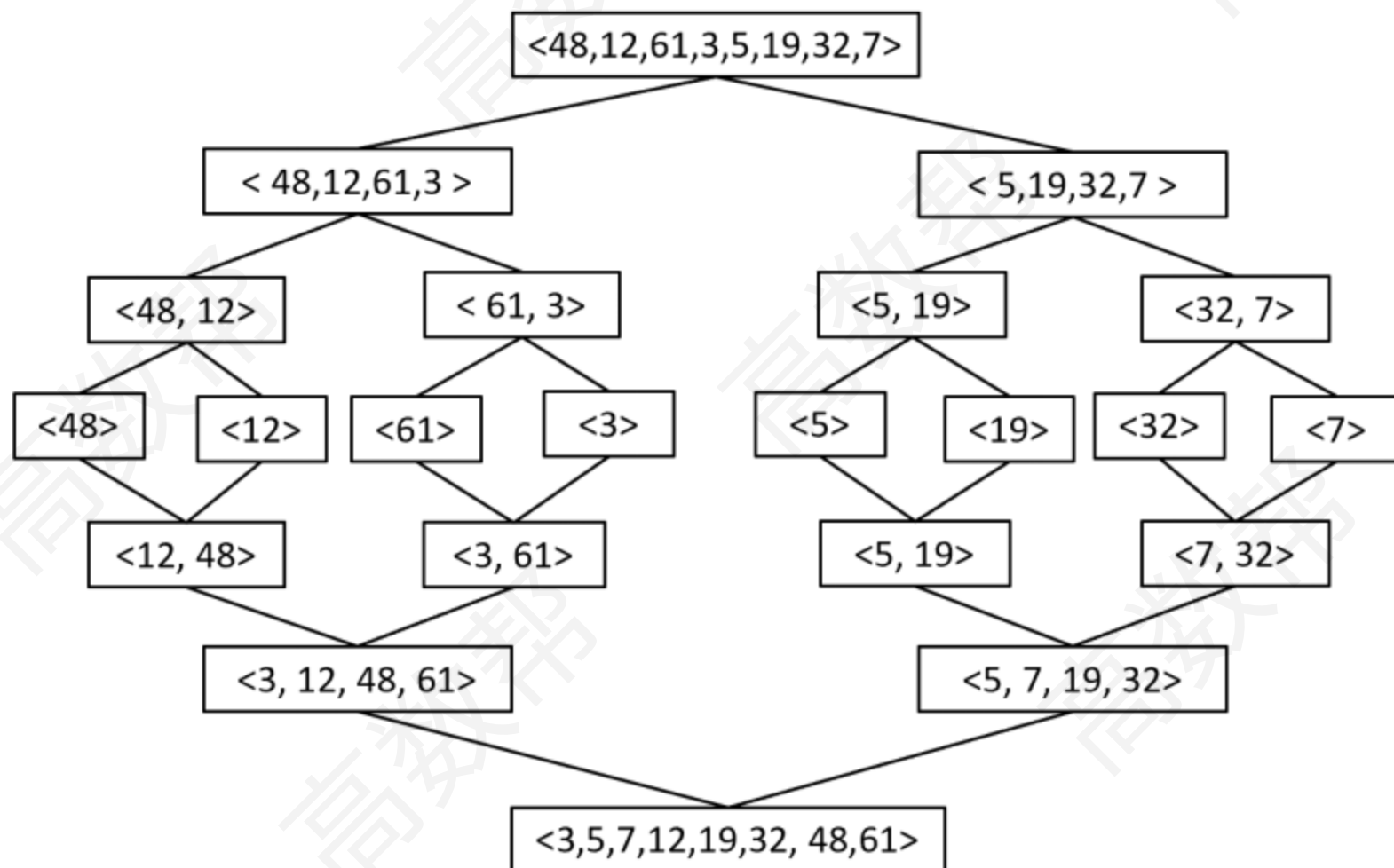
- (1)划分：将待排序 n 个元素的序列划分成两个规模为 $n/2$ 的子序列。
- (2)解决：用归并排序递归地对每一子序列排序。
- (3)合并：归并两个有序序列，得到排序结果。

2. 归并排序的时间复杂度： $O(n\log_2 n)$

【题9】 归并排序算法对下列实例排序，写出算法执行过程。

$A=(48,12,61,3,5,19,32,7)$ 。

解：



考点3 分治法应用实例（掌握）

四、快速排序算法

1.快速排序的基本思想

快速排序(Quick Sort)是分治法的另一个典型例子。

它的基本思想是：选择一个基准数，通过一趟排序将要排序的序列分割成独立的两部分；其中一部分的所有数据都比另外一部分的所有数据都要小。然后，再按此方法对这两部分数据分别进行快速排序，整个排序过程可以递归进行，直到整个序列变成有序序列。

考点3 分治法应用实例（掌握）

四、快速排序算法

2.快速排序的时间复杂度:

最坏时间复杂度: $O(n^2)$

最好时间复杂度: $O(n\log_2 n)$

平均时间复杂度: $O(n\log_2 n)$



扫码观看视频讲解更清晰

【题10】 快速排序算法对下列实例排序，算法执行过程中，写出数组A第一次被分割的过程。 $A=(2,8,7,1,3,5,6,4)$

解： 以最后一个元素4为基准数，经过一趟快速排序，把数组A划分成了两部分，即 $(2, 1, 3)$ 和 $(7, 5, 6, 8)$ ，具体排序过程如下：

2	8	7	1	3	5	6	4
---	---	---	---	---	---	---	---

2	8	7	1	3	5	6	4
---	---	---	---	---	---	---	---

2	8	7	1	3	5	6	4
---	---	---	---	---	---	---	---

2	8	7	1	3	5	6	4
---	---	---	---	---	---	---	---

2	1	7	8	3	5	6	4
---	---	---	---	---	---	---	---

2	1	3	8	7	5	6	4
---	---	---	---	---	---	---	---

2	1	3	8	7	5	6	4
---	---	---	---	---	---	---	---

2	1	3	8	7	5	6	4
---	---	---	---	---	---	---	---

2	1	3	4	7	5	6	8
---	---	---	---	---	---	---	---

考点	重要程度	占分	题型
1.动态规划基本思想	★★★★★	2-4	填空、选择、简答
2.0-1背包问题	★★★★★	2-8	填空、简答、算法设计
3.矩阵链乘问题	★★★★★	4-8	计算、算法设计
4.最长公共子序列	★★★	5-8	算法设计

考点1 动态规划基本思想（理解）

一、动态规划基本思想

动态规划法与分治法类似，也是将原问题分解成若干个子问题，先求解子问题，再从这些子问题的解得到原问题的解。

与分治法不同的是，子问题往往不是相互独立的。动态规划法所针对的问题有一个显著的特征，即它所对应的子问题树中的子问题大量重复出现。

因此动态规划法的相应**特征**是，对于重复出现的子问题，只在第一次遇到时进行求解，并把答案保存起来，以后再遇到相同子问题时直接引用，不必重新求解。

考点1 动态规划基本思想（理解）

一、动态规划基本思想

动态规划法通常用于求一个问题在某种意义下的最优解。

适合采用动态规划方法求解的问题必须具备最优子结构性质的重叠子问题性质。

当一个问题的最优解包含子问题的最优解时，则称该问题具有最优子结构性质的。

在求解一个问题的过程中，很多子问题多次重复出现，则称该问题具有重叠子问题性质。

动态规划用一个表来记录所有已解决的子问题的答案。

具体的动态规划算法尽管多种多样，但他们具有相同的填表方式。

考点1 动态规划基本思想（理解）

二、动态规划算法的设计步骤

动态规划算法的设计有4个步骤：

- (1) 刻画最优解的结构。
- (2) 递归定义最优解的值。
- (3) 以自底向上的方式计算最优解的值。
- (4) 根据计算最优值时得到的信息构造问题最优解。

步骤（1） - （3）是动态规划的基本步骤。

对于给定问题，如果只需要求出最优值，那么步骤（4）可以省略。

若需要求出问题的一个最优解，则必须执行步骤（4），步骤（3）中记录的信息是构造最优解的基础。

考点2 0-1背包问题（掌握）

一、问题描述

某商店有 n 个物品，第 i 个物品价值为 v_i ，重量（或权重）为 w_i ，背包的容量为 W 。每种物品只能选择完全装入或不装入背包，一个物品至多装入一次，因此该问题被称为0-1背包问题。目标是如何选择装入背包的物品，使装入背包的物品总价值最大。

可将0-1背包问题描述为如下形式：

$$\text{目标函数 } \max \sum_{1 \leq i \leq n} v_i x_i$$

$$\text{约束条件 } \sum_{1 \leq i \leq n} w_i x_i \leq W, x_i \in \{0, 1\}$$



扫码观看视频讲解更清晰

考点2 0-1背包问题（掌握）

二、动态规划求解0-1背包问题

1.刻画0-1背包问题最优解的结构

如果问题的一个最优解 $(x_1, x_2, \dots, x_n)$ 包含 n , 即 $x_n = 1$, 那么 $(x_1, x_2, \dots, x_{n-1})$ 一定是子问题 $1, 2, \dots, n-1$ 在背包容量为 $W - w_n$ 时的最优解。

如果这个最优解不包含物品 n , 即 $x_n = 0$, 那么这个最优解一定包含子问题 $1, 2, \dots, n-1$ 在容量为 W 时的最优解。可用反证法进行证明。

考点2 0-1背包问题（掌握）

二、动态规划求解0-1背包问题

2.递归定义最优解的值

根据子问题的最优解递归地定义问题最优解。设 $c[i, w]$ 表示背包容量为 w 时， i 个物品导致的最优解的总价值。显然原问题的最优值为 $c[n, W]$ 。

$c[i, w]$ 递归定义如下：

$$c[i, w] = \begin{cases} 0, & i=0 \text{ 或 } w=0 \\ c[i-1, w], & w_i > w \\ \max\{c[i-1, w-w_i] + v_i, c[i-1, w]\}, & i > 0 \text{ 且 } w_i \leq w \end{cases}$$



扫码观看视频讲解更清晰

考点2 0-1背包问题（掌握）

二、动态规划求解0-1背包问题

3. 计算0-1背包问题最优解的值

基于计算 $c[i, w]$ 的递归方程，利用一个表格，以自底向上的方式计算最优解的值。表格里面存储了所有子问题的最优值。

$i \backslash w$	0	1	2		W
0	$c[0,0]$	$c[0,1]$	$c[0,2]$		$c[0,W]$
1	$c[1,0]$	$c[1,1]$	$c[1,2]$		$c[1,W]$
.....					
n	$c[n,0]$	$c[n,1]$	$c[n,2]$		$c[n,W]$

考点2 0-1背包问题（掌握）

算法描述如下：

KNAPSACK-DP(n, W)

```
1   for  $w \leftarrow 0$  to  $W$ 
2       do  $c[0, w] \leftarrow 0$ 
3   for  $i \leftarrow 1$  to  $n$ 
4       do  $c[i, 0] \leftarrow 0$ 
5       for  $w \leftarrow 1$  to  $W$ 
6           do if  $w[i] \leq w$ 
7               then if  $v[i] + c[i-1, w-w[i]] > c[i-1, w]$ 
8                   then  $c[i, w] \leftarrow v[i] + c[i-1, w-w[i]]$ 
9                   else  $c[i, w] \leftarrow c[i-1, w]$ 
10          else  $c[i, w] \leftarrow c[i-1, w]$ 
```

计算最优值时间为 $O(nW)$ 。

考点2 0-1背包问题（掌握）

二、动态规划求解0-1背包问题

4.构造问题的最优解

根据最优值表，构造出0-1背包问题的最优解。

OUTPUT- SACK(c, w)

```
1   for  $i \leftarrow n$  downto 2
2       do if  $c[i, w] = c[i-1, w]$ 
3           then  $x_i \leftarrow 0$ 
4           else  $x_i \leftarrow 1$ 
5            $w \leftarrow w - w_i$ 
6    $x_1 \leftarrow c[1, w] ? 1 : 0$ 
7   return  $x$ 
```

构造最优解的时间为 $O(n)$ 。

【题1】 有5个物品，重量为 $(w_1, w_2, w_3, w_4, w_5) = (3, 4, 7, 8, 9)$ ，价值为 $(v_1, v_2, v_3, v_4, v_5) = (4, 5, 10, 11, 13)$ ，背包容量 $W=17$ ，如何选择装入物品使得背包价值最大。

解： 根据背包问题的递归方程，可以计算出背包问题的所有子问题的最优值，如下表所示：

		w																	
i	c	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
	1	0	0	0	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
	2	0	0	0	4	5	5	5	9	9	9	9	9	9	9	9	9	9	9
	3	0	0	0	4	5	5	5	10	10	10	14	15	15	15	19	19	19	19
	4	0	0	0	4	5	5	5	10	11	11	14	15	16	16	19	21	21	21
	5	0	0	0	4	5	5	5	10	11	13	14	15	17	18	19	21	23	24

根据最优值表，可得出该问题的最优解为 $(x_1, x_2, x_3, x_4, x_5) = (0, 0, 0, 1, 1)$ ，
也即背包中只放入第4和5个物品，能使背包的价值最大，为24。

考点3 矩阵链乘问题（掌握）

一、问题描述

给定 n 个矩阵 $\langle A_1, A_2, \dots, A_n \rangle$ ，其中 A_i 与 A_{i+1} 是可乘的，矩阵 A_i 的维数为 $p_{i-1} \times p_i$ ， $i=1, 2, \dots, n$ 。

如何确定矩阵连乘的计算次序，使得矩阵连乘所需的数乘次数最少。

二、完全加括号

完全加括号的矩阵连乘可递归地定义为：

- (1) 单个矩阵是完全加括号的；
 - (2) 矩阵连乘积 A 是完全加括号的，则 A 可表示为2个完全加括号的矩阵连乘积 B 和 C 的乘积并加括号，即 $A=(BC)$ 。
- 括号加在不同位置所产生的计算量是不同的。

【题2】 有四个矩阵 A, B, C, D ，它们的维数分别是： $A=50 \times 10$ ， $B=10 \times 40$ ， $C=40 \times 30$ ， $D=30 \times 5$ ，加括号方式有哪几种。

解： 连乘积 $ABCD$ 的5种加括号方式以及对应的数乘次数如下：

$$(A((BC)D)) \quad 16000$$

$$(A(B(CD))) \quad 10500$$

$$((AB)(CD)) \quad 36000$$

$$(((AB)C)D) \quad 87500$$

$$((A(BC))D) \quad 34500$$



扫码观看视频讲解更清晰

考点3 矩阵链乘问题（掌握）

三、动态规划算法求解矩阵链乘问题

将矩阵连乘 $A_i A_{i+1} \dots A_j$ 简记为 $A[i:j]$ ，这里 $i \leq j$ 。

考察计算 $A[i:j]$ 的最优计算次序。设这个次序在矩阵 A_k 和 A_{k+1} 之间将矩阵链断开， $i \leq k < j$ ，则其相应完全加括号方式为 $(A_i A_{i+1} \dots A_k)(A_{k+1} A_{k+2} \dots A_j)$ 。

$A[i:j]$ 的计算量： $(A_i A_{i+1} \dots A_k)$ 的计算量加上 $(A_{k+1} A_{k+2} \dots A_j)$ 的计算量，再加上 $(A_i A_{i+1} \dots A_k)$ 和 $(A_{k+1} A_{k+2} \dots A_j)$ 相乘的计算量。

考点3 矩阵链乘问题（掌握）

三、动态规划算法求解矩阵链乘问题

1. 分析矩阵链乘问题最优解的结构

特征: 计算 $A[i:j]$ 的最优次序所包含的计算矩阵子链 $A[i:k]$ 和 $A[k+1:j]$ 的次序也是最优的。

矩阵连乘计算次序问题的最优解包含着其子问题的最优解——最优子结构性质。

问题的最优子结构性质是该问题可用动态规划算法求解的显著特性。

矩阵链乘问题也具有重叠子问题的性质。

考点3 矩阵链乘问题（掌握）

三、动态规划算法求解矩阵链乘问题

2.建立递归关系

假设计算 $A[i:j]$, $1 \leq i \leq j \leq n$, 所需要的最少数乘次数为 $m[i:j]$, 则原问题的最优值为 $m[1:n]$ 。

又设 A_i 的维数为 $p_{i-1} \times p_i$

当 $i = j$ 时, $A[i:j] = A_i$, 因此 $m[i:j] = 0, i = 1, 2, \dots, n$

当 $i < j$ 时, $m[i:j] = m[i:k] + m[k+1:j] + p_{i-1}p_kp_j$

即可递归地定义 $m[i:j]$ 为:

$$m[i:j] = \begin{cases} 0, & i = j \\ \min_{i \leq k < j} \{m[i:k] + m[k+1:j] + p_{i-1}p_kp_j\}, & i < j \end{cases}$$



扫码观看视频讲解更清晰

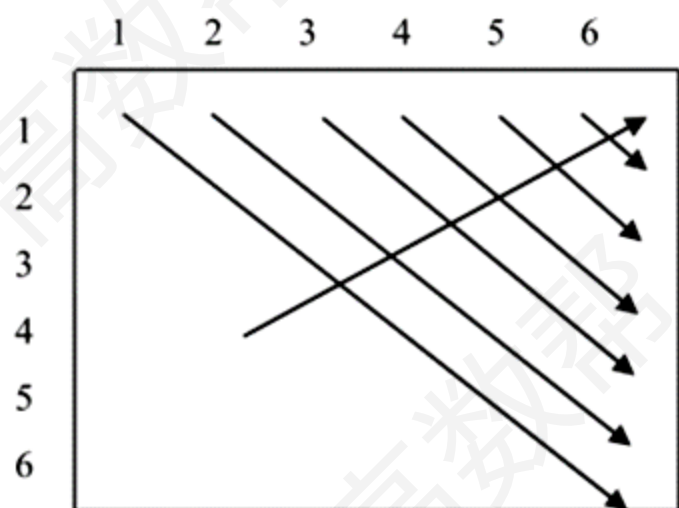
考点3 矩阵链乘问题（掌握）

三、动态规划算法求解矩阵链乘问题

3. 计算矩阵链乘的最优值

根据 $m[i:j]$ 的递归方程自底向上的求出每个子问题的最优值，把最优值填在表 m 中，最优分割位置填在表 s 中。

计算 $m[i:j]$ 的次序如下图所示。



$i \backslash j$	1	2	3		n
1	$m[1,1]$	$m[1,2]$	$m[1,3]$		$m[1,n]$
2		$m[2,2]$	$m[2,3]$		$m[2,n]$
3			$m[3,3]$		$m[3,n]$
.....					
n-1				$m[n-1,n-1]$	$m[n-1,n]$
n					$m[n,n]$

考点3 矩阵链乘问题（掌握）

例如，对于计算矩阵连乘积 $A_1A_2\dots A_5$ 的次序如下图所示

m[1][1]	m[1][2]	m[1][3]	m[1][4]	m[1][5]
	m[2][2]	m[2][3]	m[2][4]	m[2][5]
		m[3][3]	m[3][4]	m[3][5]
			m[4][4]	m[4][5]
				m[5][5]



扫码观看视频讲解更清晰

考点3 矩阵链乘问题（掌握）

算法描述如下：

MATRIX-CHAIN-ORDER(p)

```
1       $n \leftarrow \text{length}[p] - 1$ 
2      for  $i \leftarrow 1$  to  $n$ 
3          do  $m[i, i] \leftarrow 0$ 
4      for  $l \leftarrow 2$  to  $n$ 
5          do for  $i \leftarrow 1$  to  $n - l + 1$ 
6              do  $j \leftarrow i + l - 1$ 
7                   $m[i, j] \leftarrow \infty$ 
8                      for  $k \leftarrow i$  to  $j - 1$ 
9                          do  $q \leftarrow m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
10                             if  $q < m[i, j]$ 
11                                 then  $m[i, j] \leftarrow q$ 
12                                      $s[i, j] \leftarrow k$ 
13      return  $m$  and  $s$ 
```

计算最优值的时间为 $O(n^3)$ 。

考点3 矩阵链乘问题（掌握）

4. 构造问题的最优解

尽管算法给出了计算矩阵乘积的最优数乘次数（最优解的值），但仍然不知道这些矩阵相乘的次序。可从存储在 s 表中的信息构造问题的一个最优解。

OUTPUT-OPTIMAL-PARENS(s, i, j)

```
1  if  $i=j$ 
2      then output " $A$ " $i$ 
3  else output "("
4      OUTPUT-OPTIMAL-PARENS( $s, i, s[i, j]$ )
5      OUTPUT-OPTIMAL-PARENS( $s, s[i, j] + 1, j$ )
6      output ")"
```



扫码观看视频讲解更清晰

构造最优解的时间为 $O(n)$ 。

【题3】 确定计算矩阵连乘积 $A_1A_2A_3A_4A_5A_6$ 的最优次序。其中各个矩阵的维数为：

A_1	A_2	A_3	A_4	A_5	A_6
30×35	35×15	15×5	5×10	10×20	20×25

解：例如，在计算 $m[2][5]$ 时，依 $m[i][j]$ 的递归式有：

$$m[2][5] = \min \begin{cases} m[2][2] + m[3][5] + p_1 p_2 p_5 = 0 + 2500 + 35 \times 15 \times 20 = 13000 \\ m[2][3] + m[4][5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \times 5 \times 20 = 7125 \\ m[2][4] + m[5][5] + p_1 p_4 p_5 = 4375 + 0 + 35 \times 10 \times 20 = 11375 \end{cases}$$
$$= 7125$$

对应的最佳断开点 $k=3$ ，因此， $s[2][5]=3$ 。

【题3】 确定计算矩阵连乘积 $A_1A_2A_3A_4A_5A_6$ 的最优次序。其中各个矩阵的维数为：

A_1	A_2	A_3	A_4	A_5	A_6
30×35	35×15	15×5	5×10	10×20	20×25

	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

	1	2	3	4	5	6
1	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5
6						0

由此，可确定最优次序为 $(A_1(A_2A_3))((A_4A_5)A_6)$ 。

考点4 最长公共子序列 (LCS) (理解)

一、问题描述

1.子序列、公共子序列、最长公共子序列

例: $X=\langle A, B, C, B, D, A, B \rangle$

$Y=\langle B, D, C, A, B, A \rangle$

$\langle B, C, A \rangle$ 是序列X的子序列

$\langle D, C, A \rangle$ 是序列Y的子序列

$\langle B, C, B \rangle$ 是序列X和Y的公共子序列, 但不是最长公共子序列

$\langle B, C, B, A \rangle$ 和 $\langle B, D, A, B \rangle$ 都是X和Y的最长公共子序列 (长度为4)

考点4 最长公共子序列 (LCS) (理解)

一、问题描述

2.最长公共子序列问题

给定两个序列 $X = \langle x_1, x_2, \dots, x_m \rangle$ 和 $Y = \langle y_1, y_2, \dots, y_n \rangle$ ，找出 X 和 Y 的一个最长公共子序列。



扫码观看视频讲解更清晰

考点4 最长公共子序列 (LCS) (理解)

二、动态规划算法求解最长公共子序列问题

给定序列 $X = \langle x_1, x_2, \dots, x_m \rangle$ ，定义 $X_i = \langle x_1, x_2, \dots, x_i \rangle$ 为 X 的第 i 个前缀，其中 $i = 1, 2, \dots, m$ 。例如，如果 $X = \langle B, A, C, A, D, A, B \rangle$ ，则 $X_3 = \langle B, A, C \rangle$ ， X_0 为空序列。

1. 刻画LCS问题的最优子结构

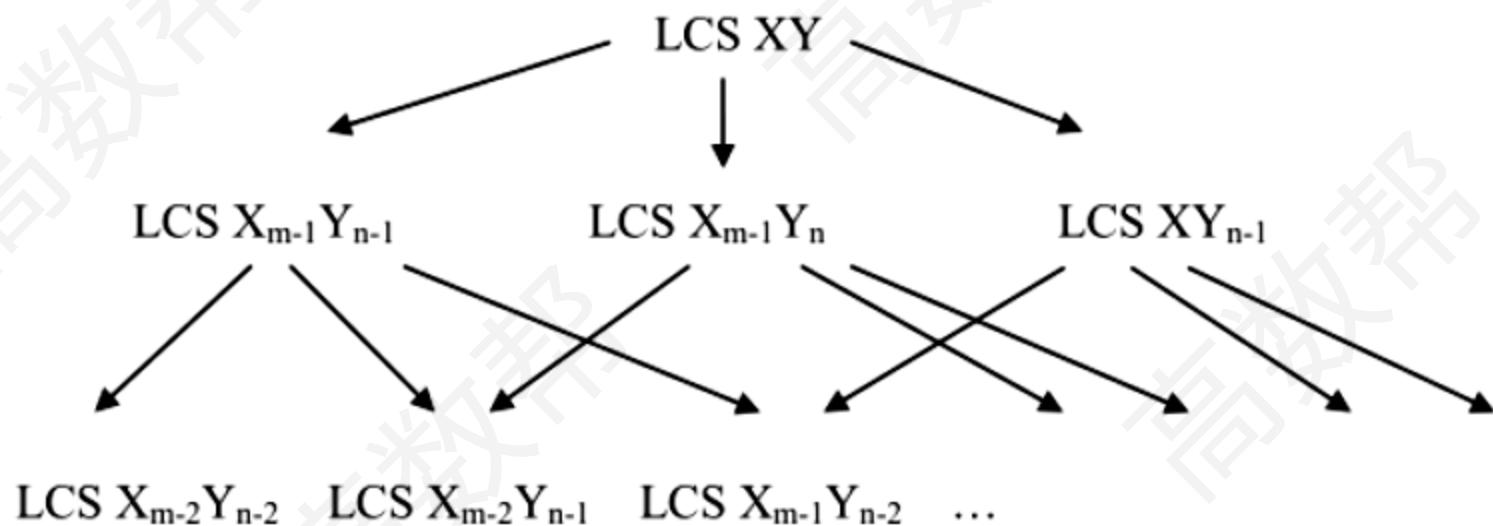
设 $X = \langle x_1, x_2, \dots, x_m \rangle$ 和 $Y = \langle y_1, y_2, \dots, y_n \rangle$ 是两条序列， $Z = \langle z_1, z_2, \dots, z_k \rangle$ 是 X 和 Y 的任一最长公共子序列。

- (1) 如果 $x_m = y_n$ ，那么 $z_k = x_m = y_n$ ，且 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的一条最长公共子序列。
- (2) 如果 $x_m \neq y_n$ 且 $z_k \neq x_m$ ，那么 Z 是 X_{m-1} 和 Y 的一条最长公共子序列。
- (3) 如果 $x_m \neq y_n$ 且 $z_k \neq y_n$ ，那么 Z 是 X 和 Y_{n-1} 的一条最长公共子序列。

考点4 最长公共子序列 (LCS) (理解)

这个定理告诉我们，两个序列的最长公共子序列包含了这两个序列的前缀的最长公共子序列。

因此，最长公共子序列问题具有最优子结构性质，其递归解具有重叠子问题性质。所以可用动态规划算法解此问题。



考点4 最长公共子序列 (LCS) (理解)

2.递归定义LCS问题最优解的值

设 $l[i, j]$ 表示序列 X_i 和 Y_j 的LCS的长度, 如果 $i=0$ 或 $j=0$, 则其中有一个序列长度为0, 因此, LCS长度为0。由LCS问题的最优子结构可导出以下递归公式:

$$l[i, j] = \begin{cases} 0 & , \quad i = 0 \text{ 或 } j = 0, \\ l[i - 1, j - 1] + 1 & , \quad i, j > 0 \text{ 且 } x_i = y_j \\ \max(l[i, j - 1], l[i - 1, j]) & , \quad i, j > 0 \text{ 且 } x_i \neq y_j \end{cases}$$

考点4 最长公共子序列 (LCS) (理解)

3. 计算LCS最优解的值

利用自底向上的动态规划求解这一问题。将 $l[i, j]$ 的值存储在表 $l[0 \cdots m, 0 \cdots n]$ 中，以行为主序，从左到右计算表 $l[0 \cdots m, 0 \cdots n]$ 中的元素，同时维持表 $b[0 \cdots m, 0 \cdots n]$ ，以简化最优解的构造。

当计算 $l[i, j]$ 时，用 $b[i, j]$ 记录使得 $l[i, j]$ 取最优值的最优子问题。

考点4 最长公共子序列 (LCS) (理解)

算法描述如下:

LCS-LENGTH(X, Y)

```
1       $m \leftarrow \text{length}[X]$ 
2       $n \leftarrow \text{length}[Y]$ 
3      for  $i \leftarrow 1$  to  $m$ 
4          do  $l[i, 0] \leftarrow 0$ 
5      for  $j \leftarrow 1$  to  $n$ 
6          do  $l[0, j] \leftarrow 0$ 
7      for  $i \leftarrow 1$  to  $m$ 
8          do for  $j \leftarrow 1$  to  $n$ 
9              do if  $x_i = y_j$ 
10                 then  $l[i, j] \leftarrow l[i-1, j-1] + 1$ 
11                      $b[i, j] \leftarrow \text{"\textbackslash"}$ 
12                 else if  $l[i-1, j] \geq l[i, j-1]$ 
13                     then  $l[i, j] \leftarrow l[i-1, j]$ 
14                          $b[i, j] \leftarrow \text{"\textuparrow"}$ 
15                     else  $l[i, j] \leftarrow l[i, j-1]$ 
16                          $b[i, j] \leftarrow \text{"\textless"}$ 
17      return  $l$  and  $b$ 
```

计算最优值的时间为 $O(mn)$ 。

考点4 最长公共子序列 (LCS) (理解)

4.构造 X 和 Y 的一条 LCS

表 $b[0 \cdots m, 0 \cdots n]$ 可用于快速构造 $X = \langle x_1, x_2, \cdots, x_m \rangle$ 和 $Y = \langle y_1, y_2, \cdots, y_n \rangle$ 的一条 LCS 。从 $b[m, n]$ 开始, 在表中沿着箭头的方向跟踪。

当出现 $\nwarrow$ 时, 表示 $x_i = y_j$ 为 LCS 中的元素。

因此, 这个方法按照逆序得到 LCS 中的元素。



扫码观看视频讲解更清晰

考点4 最长公共子序列 (LCS) (理解)

OUTPUT-LCS(b, X, i, j)

```
1      if  $i = 0$  or  $j = 0$ 
2          then return
3      if  $b[i, j] = \text{"↖"}$ 
4          then OUTPUT-LCS( $b, X, i-1, j-1$ )
5          output  $x_i$ 
6      else if  $b[i, j] = \text{"↑"}$ 
7          then OUTPUT-LCS( $b, X, i-1, j$ )
8      else OUTPUT-LCS( $b, X, i, j-1$ )
```

构造最优解的时间为 $O(m+n)$ 。

【题4】 有两个给定序列 $X = \langle A, B, C, B, D, A, B \rangle$ ， $Y = \langle B, D, C, A, B, A \rangle$ ，找出这两个序列的最长公共子序列。

解： 根据最长公共子序列问题的递归方程，可得出所有子问题的最优值如下图

(b) 所示：

		j	0	1	2	3	4	5	6
i	l	Y_j	B	D	C	A	B	A	
		X_i							
0	X_i	0	0	0	0	0	0	0	
1	A	0							
2	B	0							
3	C	0							
4	B	0							
5	D	0							
6	A	0							
7	B	0							

(a) 初始化

		j	0	1	2	3	4	5	6
i	l	Y_j	B	D	C	A	B	A	
		X_i							
0	X_i	0	0	0	0	0	0	0	
1	A	0	↑0	↑0	↑0	↘1	←1	↘1	
2	B	0	↘1	←1	←1	↑1	↘2	←2	
3	C	0	↑1	↑1	↘2	←2	↑2	↑2	
4	B	0	↘1	↑1	↑2	↑2	↘3	←3	
5	D	0	↑1	↘2	↑2	↑2	↑3	↑3	
6	A	0	↑1	↑2	↑2	↘3	↑3	↘4	
7	B	0	↘1	↑2	↑2	↑3	↘4	↑4	

(b) X和Y的最优LCS值

从 $b[7,6]$ 出发，沿着箭头方向跟踪，可以找到X和Y的最长公共子序列为 $\langle B, C, B, A \rangle$ 。



考点	重要程度	占分	题型
1. 贪心法的基本思想	★★★	2-5	填空、简答
2. 背包问题	★★★★	2-8	填空、算法设计
3. 活动选择问题	★★★	2-8	填空、算法设计
4. 贪心法的其他应用	★★★★	2-8	填空、简答、算法设计



扫码观看视频讲解更清晰

考点1 贪心法基本思想（理解）

1、贪心法的基本思想

求解最优化问题的贪心算法包含一系列步骤，每一步都在一组选择中做出在当前看来最好的选择，希望通过做出局部最优选择达到全局最优选择。但贪心算法不一定总产生最优解，所以一个贪心算法是否产生最优解，需要严格证明。

贪心法求解的问题必须具有**最优子结构性质**和**贪心选择性质**。

考点1 贪心法基本思想（理解）

2、贪心法和动态规划法的区别

共同点： 贪心算法和动态规划算法都要求问题具有最优子结构性质，这是这两类算法的一个共同点。

区别： 动态规划算法中，每步所做的选择往往依赖于相关子问题的解，因而只有在解出所有相关子问题时才能做出选择。而贪心算法，仅在当前状态下做出最好选择，即局部最优选择，然后再去解做出这个选择后产生的相应的子问题。

考点2 背包问题（掌握）

1、问题描述

某商店有 n 个物品，第 i 个物品价值为 v_i ，重量（或权重）为 w_i ，背包的容量为 W ，目标是如何选择装入背包的物品，使装入背包的物品总价值最大。在选择物品装入背包时，可以只选择物品的一部分，而不一定要选择物品的全部。可将这个问题形式描述如下：

目标函数
$$\max \sum_{1 \leq i \leq n} v_i x_i$$

约束条件
$$\sum_{1 \leq i \leq n} w_i x_i \leq W, x_i \in [0, 1]$$



扫码观看视频讲解更清晰

考点2 背包问题（掌握）

背包问题具有最优子结构性质和重叠子问题性质，它可以用动态规划算法来解。

但用贪心算法更简单，解题效率更高。

贪心算法总是做出在当前看来是最好的选择。

也就是说贪心算法并不从整体最优上加以考虑，所以贪心算法并不是对所有问题都能得到整体最优解。

考点2 背包问题（掌握）

2、贪心策略和最优子结构

背包问题的贪心策略是优先选择单位重量价值 v_i / w_i 最大的物品放入背包。

每次从物品集合中选择单位重量价值最大即价值率最大的物品，如果其重量小于背包容量，就可以把它装入，并将背包价值增加该物品的价值，同时背包容量减去该物品的重量，然后就面临了一个最优子问题——它同样是背包问题，只不过背包容量减少了，物品集合减少了。

因此可以证明背包问题具有最优子结构性质。

考点2 背包问题（掌握）

3、0-1背包问题和背包问题

0-1 背包问题也具有最优子结构性质。

但是 0-1 背包问题却不能用贪心算法求解。

对于 0-1 背包问题，贪心选择之所以不能得到最优解是因为在这种情况下，它无法保证最终将背包装满，背包部分空间的闲置使每单位重量背包空间所具有的价值降低了(该问题不具有贪心选择性)。

考点2 背包问题（掌握）

4、算法描述

GREEDY-KNAPSACK(v, w, W, n)

```
1       $x \leftarrow 0$            //初始化解
2       $c \leftarrow W$        //  $c$ : 背包剩余可用容量
3      for  $i \leftarrow 1$  to  $n$  do
4          if  $w(i) \leq c$     //当前考虑的物品权值小于背包现有容量
5              then  $x(i) \leftarrow 1$  //物品放入
6                   $c \leftarrow c - w(i)$  //更新现有背包容量
7          else break
8      if  $i \leq n$ 
9          then  $x(i) \leftarrow c/w(i)$  //将物品一部分放入背包
10     return  $x$            //返回背包问题解
```



扫码观看视频讲解更清晰

考点2 背包问题（掌握）

算法复杂性分析：

n 个物品已按照价值率非增次序排列： $T(n) = \theta(n)$;

n 个物品未按照价值率非增次序排列： $T(n) = \theta(n) + \theta(n \log n) = \theta(n \log n)$ 。

算法正确性证明：

- ① 证明问题具有最优子结构性质;
- ② 证明问题具有贪心选择性质。

【题1】 有背包问题如下： $n=6$ ， $W=20$ ， $(v_1, v_2, \dots, v_6) = (4, 8, 15, 1, 6, 3)$ ，
 $(w_1, w_2, \dots, w_6) = (5, 3, 2, 10, 4, 8)$ 。其中 n 为物品个数， W 为背包载重量，
 v 表示物品的价值， w 表示物品的重量。请问对于此背包问题，应如何选择放
进去的物品，才能使到放进背包的物品总价值最大。

解： 把物品按照价值率非增次序进行排列，如下表

i	1	2	3	4	5	6
w_i	2	3	4	5	8	10
v_i	15	8	6	4	3	1
v_i/w_i	7.5	2.67	1.5	0.8	0.375	0.1

以选择价值率高的物品为贪心策略。

① 先把重量为2的物品放进背包，此时背包剩余容量为18，背包价值 V 为15；

【题1】 有背包问题如下： $n=6$ ， $W=20$ ， $(v_1, v_2, \dots, v_6) = (4, 8, 15, 1, 6, 3)$ ，
 $(w_1, w_2, \dots, w_6) = (5, 3, 2, 10, 4, 8)$ 。其中 n 为物品个数， W 为背包载重量，
 v 表示物品的价值， w 表示物品的重量。请问对于此背包问题，应如何选择放
进去的物品，才能使到放进背包的物品总价值最大。

- ② 再把重量为3的物品放进背包，此时背包剩余容量为15，背包价值 V 为23；
- ③ 再把重量为4的物品放进背包，此时背包剩余容量为11，背包价值 V 为29；
- ④ 再把重量为5的物品放进背包，此时背包剩余容量为6，背包价值 V 为33；
- ⑤ 最后把第5个物品的 $6/8$ 放进去，即第5个物品放进去的重量为6，此时背包
已满，背包价值为35.25。

结果是把排序后的前四个物品装进背包，第5个物品装入重量6，背包总价值
最大，为35.25。

考点3 活动选择问题（理解）

1、问题描述

活动选择问题即若干个具有竞争性的活动要求互斥使用某一公共资源，目标是选择最大相容活动子集。

假定集合 $S = \{a_1, a_2, \dots, a_n\}$ 中含有 n 个希望使用某一资源的活动，比如教室、某一设备等，它们一次只能由一个活动使用。每个活动 a_i 有开始时间 s_i 和完成时间 f_i ，其中 $0 \leq s_i < f_i < \infty$ 。如果某个活动 a_i 被选中使用资源，则该活动在半开区间 $(s_i, f_i]$ 这段时间内占据资源。如果活动 a_i 和 a_j 在时间区间 $(s_i, f_i]$ 和 $(s_j, f_j]$ 上不重叠，则称它们是相容的，即如果 $s_i \geq f_j$ 或者 $s_j \geq f_i$ ，活动 a_i 和 a_j 是相容的。

考点3 活动选择问题（理解）

2、贪心策略和最优子结构

活动选择问题为了选择最多的相容活动，每次选择完成时间最小的活动，使剩余的可安排时间段极大化，以便接待尽可能多的相容活动。

定义活动选择问题的子空间 $S_{ij} = \{a_k \in S : f_i \leq s_k < f_k \leq s_j\}$ ， S_{ij} 表示活动集合 S 的一个子集，表示在活动 a_i 完成之后开始，在活动 a_j 开始之前完成的所有活动集。

活动选择问题的最优子结构描述如下：假定 S_{ij} 的最优解 A_{ij} 包括 a_k ，那么 S_{ik} 的解 A_{ik} 和 S_{kj} 的解 A_{kj} 也是最优的。可用反证法进行证明。

考点3 活动选择问题（理解）

3、算法描述

ITERATIVE-ACTIVITY-SELECT (s, f)

```
1       $n \leftarrow \text{length}[s]$ 
2       $A \leftarrow \{a_1\}$            //活动 $a_1$ 加入集合
3       $i \leftarrow 1$ 
4      for  $m \leftarrow 2$  to  $n$ 
5          do if  $s_m \geq f_i$        //找到满足条件的活动 $m$ 
6              then  $A \leftarrow A \cup \{a_m\}$  //将活动 $m$ 加入集合
7                   $i \leftarrow m$    //更新当前加入的活动
8      return  $A$ 
```

考点3 活动选择问题（理解）

算法复杂性分析：

当活动按结束时间已排序： $T(n) = \theta(n)$;

当活动按结束时间未排序： $T(n) = \theta(n) + \theta(n \log n) = \theta(n \log n)$ 。

算法正确性证明：

- ① 证明问题具有最优子结构性质;
- ② 证明问题具有贪心选择性质。



扫码观看视频讲解更清晰

【题2】 设活动集 $S=\{1, 2, \dots, 11\}$ 中的活动已按完成时间非减序排列，找出最大相容活动子集。

活动 i	1	2	3	4	5	6	7	8	9	10	11
开始时间 s_i	1	3	0	5	3	5	6	8	8	2	12
结束时间 f_i	4	5	6	7	8	9	10	11	12	13	14

解：贪心策略是选择完成时间最小的活动，具体步骤如下：

第1步，选择活动1；

第2步，在剩余相容活动子集 $\{4, 6, 7, 8, 9, 11\}$ 中选结束时间最早的活动4；

第3步，在剩余相容活动子集 $\{8, 9, 11\}$ 中选结束时间最早的活动8；

第4步，在剩余相容活动子集 $\{11\}$ 中选结束时间最早的活动11。

这时，剩余相容活动子集 $\{\}$ 变为空集，从而得 S 的一个最优解 $A=\{1, 4, 8, 11\}$ 。

【题3】 某体育馆有一羽毛球场出租，现在总共有 10 位客户申请租用此羽毛球场，每个客户所租用的时间单元如下表所示， $s(i)$ 表示开始租用时刻， $f(i)$ 表示结束租用时刻，10 个客户的申请如下表所示：

i	1	2	3	4	5	6	7	8	9	10
$s(i)$	0	3	1	5	3	5	11	8	8	6
$f(i)$	6	5	4	9	8	7	13	12	11	10

同一时刻，该羽毛球场只能租借给一位客户，请设计一个租用安排方案，在这10位客户里面，使得体育馆能尽可能满足多位客户的需求，并算出针对上表的10个客户申请，最多可以安排几位客户申请。

解：将这 10 位客户的申请按照结束时间 $f(i)$ 递增排序，如下表：

i	1	2	3	4	5	6	7	8	9	10
$s(i)$	1	3	0	5	3	5	6	8	8	11
$f(i)$	4	5	6	7	8	9	10	11	12	13

【题3】 某体育馆有一羽毛球场出租，现在总共有 10 位客户申请租用此羽毛球场，每个客户所租用的时间单元如下表所示， $s(i)$ 表示开始租用时刻， $f(i)$ 表示结束租用时刻，10 个客户的申请如下表所示：

i	1	2	3	4	5	6	7	8	9	10
$s(i)$	0	3	1	5	3	5	11	8	8	6
$f(i)$	6	5	4	9	8	7	13	12	11	10

同一时刻，该羽毛球场只能租借给一位客户，请设计一个租用安排方案，在这10位客户里面，使得体育馆能尽可能满足多位客户的需求，并算出针对上表的10个客户申请，最多可以安排几位客户申请。

贪心策略是选择结束租用时间最小的申请，具体步骤如下：

- ① 选择申请1 (1,4) ；
- ② 依次检查后续客户申请，只要与已选择的申请相容或不冲突，则选择该申请。直到所有申请检查完毕。选择申请4 (5,7) 、申请8 (8,11) 、申请10 (11,13) ；
- ③ 最后，选择申请1 (1,4) 、申请4 (5,7) 、申请8 (8,11) 、申请10 (11,13) 共4个客户申请。这已经是可以满足的最大客户人数。

考点4 贪心法的其他应用（掌握）

1、哈夫曼编码

哈夫曼编码是数据压缩常用的方法，是一种变长编码方式，比起定长编码的 *ASCII* 编码来说，哈夫曼编码能节省很多的空间。

哈夫曼编码首先会使用字符的频率创建一棵树，然后通过这个树的结构为每个字符生成一个特定的编码，出现频率高的字符使用较短的编码，出现频率低的则使用较长的编码，这样就会使编码之后的字符串平均长度降低，从而达到数据无损压缩的目的。

哈夫曼编码是利用贪心法实现的，即基于频率最小构造哈夫曼树。哈夫曼编码也具有贪心选择性质和最优子结构性质。

【题4】 已知字符和在序列中出现的频率，试构造哈夫曼树。

字符	a	b	c	d	e	f
出现频率	45 000	13 000	12 000	16 000	9000	5000

解：贪心策略是基于频率最小策略。

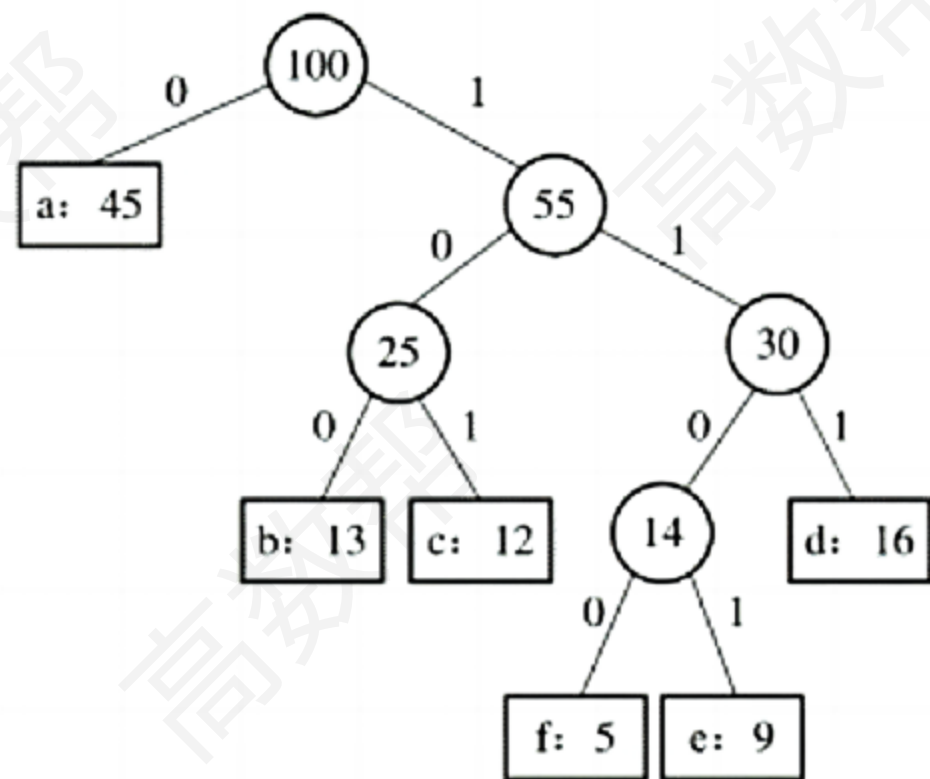
按照频率递增进行排序

$f:5 \rightarrow e:9 \rightarrow c:12$

$b:13 \rightarrow d:16 \rightarrow a:45$

最终6个字符的编码方式为：

$a\ 0, b\ 100, c\ 101, d\ 111, e\ 1101, f\ 1100。$



考点4 贪心法的其他应用（掌握）

2、最小生成树

在一给定的无向图 $G = (V, E)$ 中， (u, v) 代表连接顶点 u 与顶点 v 的边，而 $w(u, v)$ 代表此边的权重，若存在 T 为 E 的子集且为无环图，使得联通所有结点的 $w(T)$ 最小，则此 T 为 G 的最小生成树。

求最小生成树的Prim算法和Kruskal算法，都是基于贪心算法的，贪心策略是选择路径最小的边，同时最小生成树也满足最优子结构性质。

考点4 贪心法的其他应用（掌握）

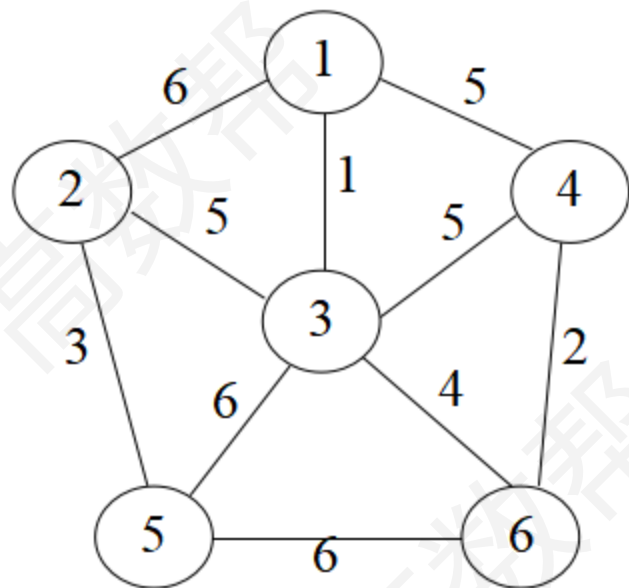
*Prim*算法求最小生成树的步骤:

- (1) 选取权值最小边的其中一个顶点作为起始点。
- (2) 找到离当前顶点权值最小的边，并记录该顶点为已选择。
- (3) 重复第二步，直到找到所有顶点，就找到了图的最小生成树。

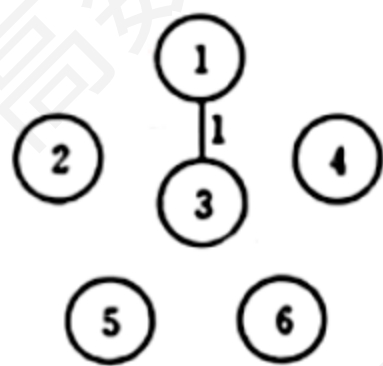
*Kruskal*算法的基本步骤如下:

- (1) 将所有边都按权值从小到大排序。
- (2) 按排序后的顺序选择权值最小的边 (u,v) 。
- (3) 如果节点 u 和 v 属于两个不同的连通分支，则将边 (u,v) 加入边集 TE 中，并将两个连通分支合并。
- (4) 如果选取的边数小于 $n-1$ ，则转向步骤2，否则算法结束。

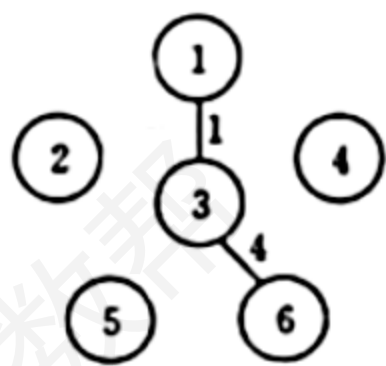
【题5】 对于下图所示的带权图，给出按照 $Prim$ 算法构造其最小生成树的过程。



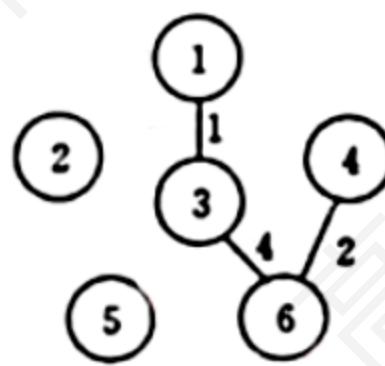
解：



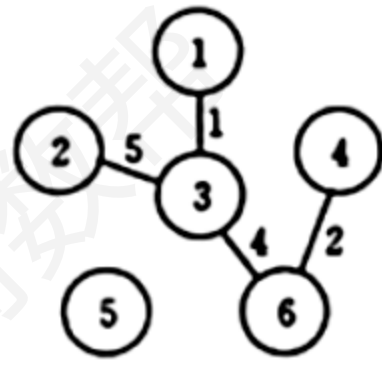
(a)



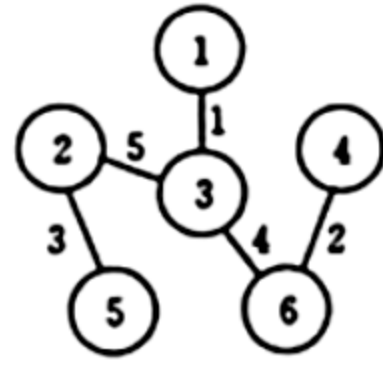
(b)



(c)

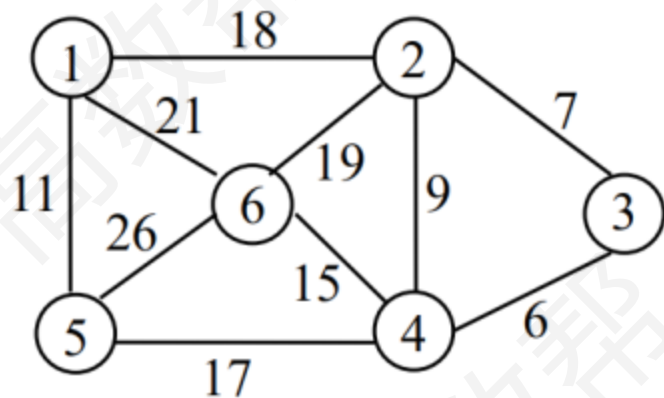


(d)



(e)

【题6】 对下图所示的连通网络 G ，用克鲁斯卡尔(Kruskal)算法求 G 的最小生成树 T ，请写出在算法执行过程中，依次加入 T 的边集 TE 中的边。说明该算法的贪心策略和算法的基本思想，并简要分析算法的时间复杂度。



解： $(3,4), (2,3), (2,4), (1,5), (4,6), (4,5), (1,2), (2,6), (1,6), (5,6)$

$TE = \{(3,4), (2,3), (1,5), (4,6), (4,5)\}$

贪心策略是每次都在连接两个不同连通分量的边中选权值最小的边。

基本思想：首先将图中所有顶点都放到生成树中，然后每次都在连接两个不同连通分量的边中选权值最小的边，将其放入生成树中，直到生成树中有 $n-1$ 条边。

时间复杂度为： $O(eln e)$



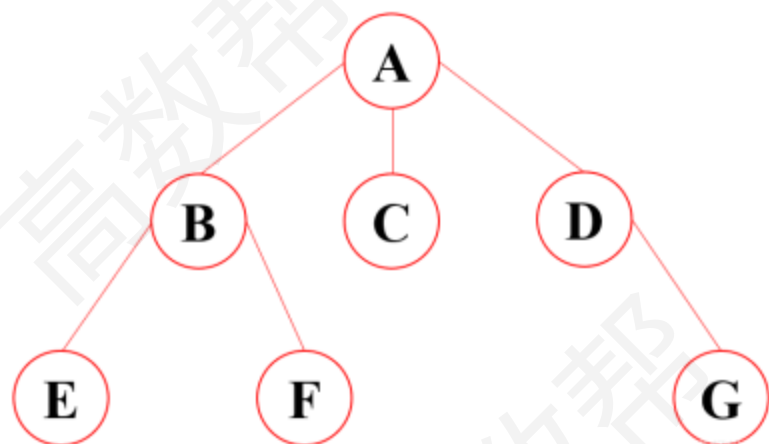
考点	重要程度	占分	题型
1. 回溯法的基本思想	★★★★★	2-4	填空、判断、简答
2. n皇后问题	★★★★★	2-5	填空、简答
3. 0-1背包问题	★★★★	2-8	填空、判断、简答
4. 着色问题	★★★★★	2-8	填空、简答、算法设计

考点1 回溯法的基本思想（掌握）

回溯法是一个既带有系统性又带有跳跃性的搜索法。为了实现回溯，首先需要为问题定义一个解空间，这个空间必须至少包含问题的一个解(可能是最优的)。在一般的情况下，会将问题的解表示成一个向量 $X=(x_1, x_2, \dots, x_n)$ ，其中元素 $x_i \in S_i$ 。通常，需要找出满足某一约束条件且使某一规范函数取最大值(或最小值)的解向量。

回溯法在包含问题所有解的解空间树中，按照深度优先的策略，从根结点出发进行搜索。搜索每到达解空间树的一个结点，总是先判断以该结点为根的子树（或结点）是否肯定不包含问题的解。如果肯定不包含，则跳过对该子树的系统搜索，一层一层地向它的祖先结点回溯，直到遇上一个结点，该节点还有儿子结点未被搜索过，此时转向该结点的一个未曾搜索过的儿子结点，继续搜索；否则，进入该子树，继续按深度优先的策略进行搜索。

考点1 回溯法的基本思想



回溯法在用来求问题的所有解（或最优解）时，要回溯到根，且根的所有儿子都已被搜索过才结束；而在用来求问题的任一解时，只要搜索到问题的一个解就可结束。

回溯法的搜索特点：在解空间树上跳跃式地深度优先搜索，即用判定函数考察 x_k 的取值，如果 x_k 是合理的就搜索 x_k 为根结点的子树，如果 x_k 取完了所有的值，便回溯到 x_{k-1} 。

考点1 回溯法的基本思想（掌握）

问题解空间的构造方式：

子集树：当所给问题是从 n 个元素的集合中找出满足某种性质的子集时，相应的解空间树称为子集树。例如：0—1背包问题。

排列树：当所给的问题是从 n 个元素的集合中找出满足某种性质的排列时，相应的解空间树称为排列树。例如： n 皇后问题。



扫码观看视频讲解更清晰

考点2 n 皇后问题（掌握）

1、问题描述

在 $n \times n$ 的棋盘上摆放 n 个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行同一列或同一斜线上。下图表示了8皇后问题的一个解。

	1	2	3	4	5	6	7	8
1				Q				
2						Q		
3								Q
4		Q						
5							Q	
6	Q							
7			Q					
8					Q			

考点2 n 皇后问题（掌握）

1、问题描述

对 n 皇后问题，可用 n 元组 $(x_1, x_2, \dots, x_n)$ 表示它的解。其中， x_i 表示皇后 i 放在棋盘第 i 行的第 x_i 列。由于不允许将两个皇后放在同一列，所以解向量中的 x_i 互不相同。

对 $n \times n$ 格棋盘的行和列分别从上到下和从左到右编号为 $1, 2, 3, \dots, n$ ，从棋盘左上角到右下角的主对角线及其平行线（即斜率为 $+1$ 的各条斜线）上，每个格的两个坐标的差值（行号-列号）相等；从棋盘右上角到左下角的主对角线及其平行线（即斜率为 -1 的各条斜线）上，每个格的两个坐标的和值（行号+列号）相等。

考点2 n 皇后问题（掌握）

所以，若第 i 个皇后和第 k 个皇后放置的位置分别是 (i, x_i) 和 (k, x_k) 则约束条件为：

$x_i \neq x_k$ (不在同一列)

$i - x_i \neq k - x_k$, 且 $i + x_i \neq k + x_k$ (不在同一斜线), 即 $|i - k| \neq |x_i - x_k|$

用回溯法解 n 皇后问题时，可用限界函数 $PLACE$ 剪去不满足行、列和斜线约束的子树，以深度优先搜索方式递归地对可行子树进行搜索。

考点2 n 皇后问题（掌握）

3、算法描述

N-QUEEN(n)

```
1       $x[1] \leftarrow 0$                                 //第1个皇后的列位置初始化
2       $k \leftarrow 1$                                 //当前行
3      while  $k > 0$  do
4           $x[k] \leftarrow x[k] + 1$                     //到下一列
5          while  $x[k] \leq n$  & not PLACE( $k$ ) do
6               $x[k] \leftarrow x[k] + 1$ 
7          if  $x[k] \leq n$                                 //找到一个位置
8              then if  $k = n$                             //测试是否为问题的解
9                  then output(X) //输出解
10             else  $k \leftarrow k + 1$  //转下一行，即给下一个皇后找位置
11                  $x[k] \leftarrow 0$  //初始化当前皇后列取值
12             else  $k \leftarrow k - 1$                 //回溯
13      return
```

考点2 n 皇后问题（掌握）

3、算法描述

PLACE(k)

1 $i \leftarrow 1$

2 **while** $i < k$ **do**

3 **if** $(x[i] = x[k] \text{ or } \text{abs}(x[i] - x[k]) = \text{abs}(i - k))$ //同一列或同一
 对角线有两个皇后

4 **then return false**

5 $i \leftarrow i + 1$

6 **return true**

n 皇后问题回溯算法的判别函数PLACE的基本流程是：将第 k 行的皇后分别与前 $k-1$ 行的皇后比较，看是否与它们相容，如果不相容就返回 *false*，否则返回 *true*。

【题1】 分析4皇后问题。

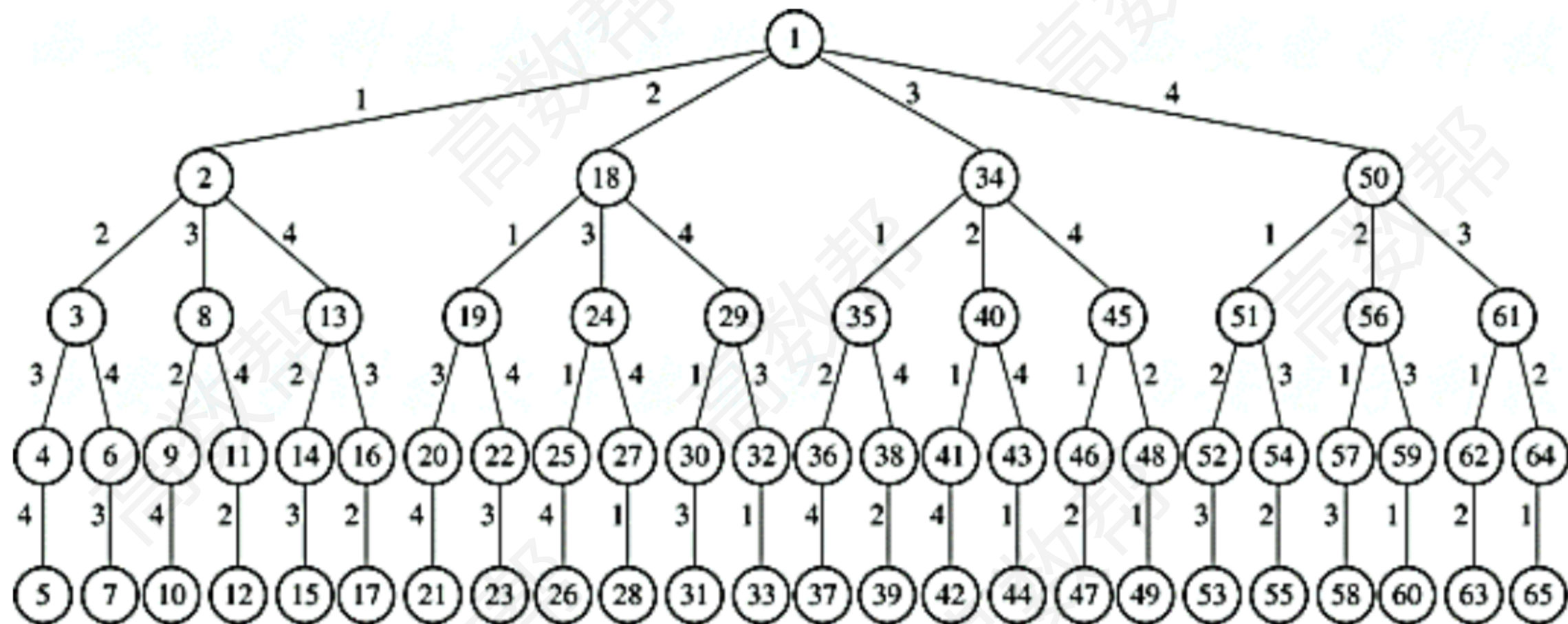


图1 4皇后问题的解空间树

【题1】 分析4皇后问题。

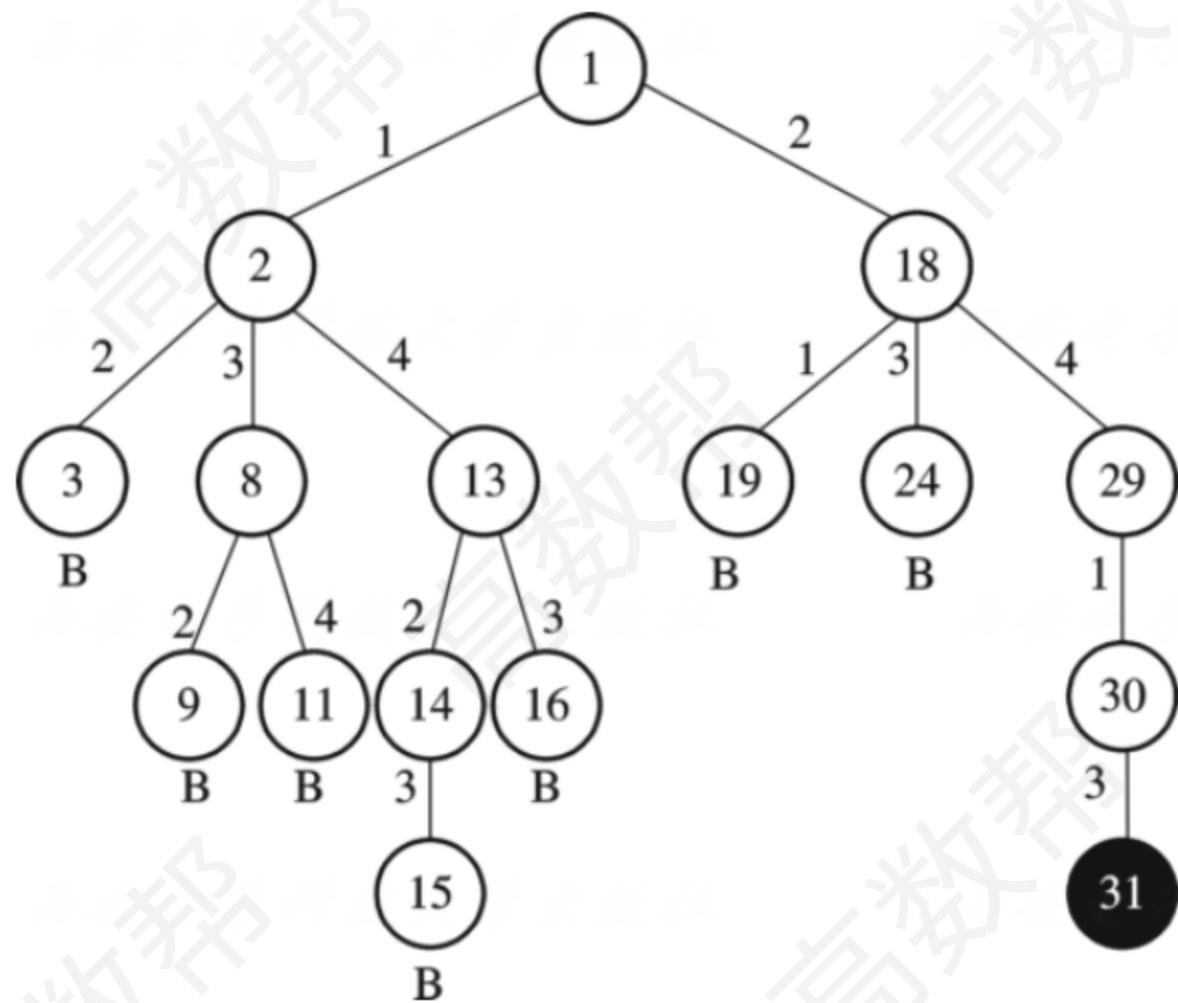


图2 具有有限界函数的4皇后问题的部分状态空间树

考点3 0-1背包问题（理解）

1、问题描述

某商店有 n 个物品，第 i 个物品价值为 v_i ，重量(或权重)为 w_i ，背包的容量为 W 。

每种物品只能选择完全装入或不装入背包，一个物品至多装入一次，因此该问题被称为0-1背包问题。目标是如何选择装入背包的物品，使装入背包的物品总价值最大。可将这个问题形式描述如下：

$$\text{目标函数 } \max \sum_{1 \leq i \leq n} v_i x_i$$

$$\text{约束条件 } \sum_{1 \leq i \leq n} w_i x_i \leq W, x_i \in \{0, 1\}$$



扫码观看视频讲解更清晰

考点3 0-1背包问题（理解）

2、问题分析

(1) 问题的解空间

对于有 n 种可选物品的0-1背包问题，其解空间由长度为 n 的0-1向量组成。该解空间包含了对变量的所有可能的0-1赋值。当 $n=3$ 时，其解空间是： $\{(0,0,0), (0,1,0), (0,0,1), (1,0,0), (0,1,1), (1,0,1), (1,1,0), (1,1,1)\}$ 。

考点3 0-1背包问题（理解）

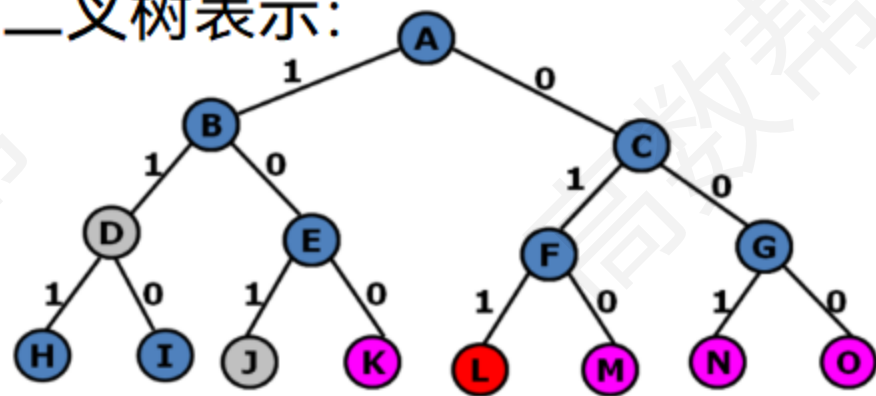
2、问题分析

(2)问题的解空间树

对于 $n=3$ 时的0-1背包问题，其解空间用一棵完全二叉树表示：

物品个数 $n=3$ 的 0-1 背包问题。

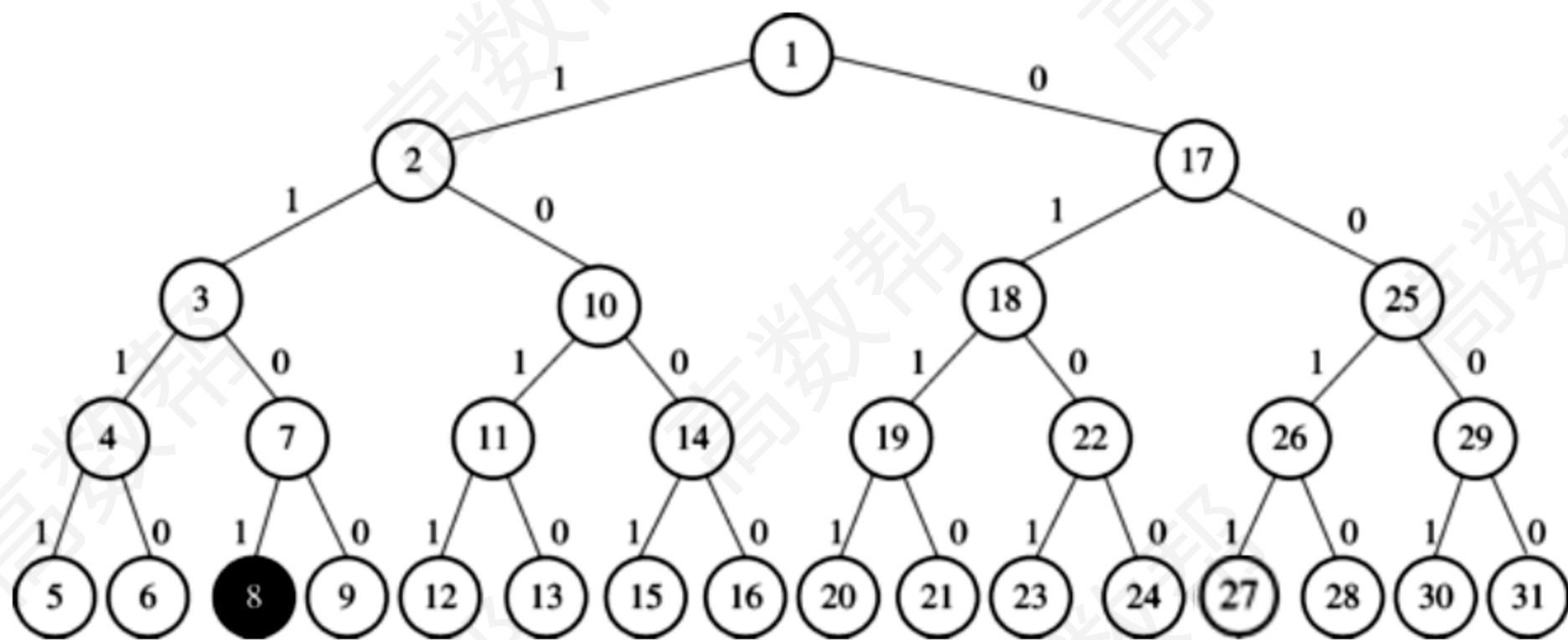
$$w=[16,15,15], v=[45,25,25], W=30。$$



图中的结点 K 、 L 、 M 、 N 、 O 分别对应一个可行解，结点 K 相应的解的价值是45，结点 L 相应的解价值是50，结点 M 相应的解价值是25，结点 N 相应的解价值是25，结点 O 相应的解价值是0；结点 L 对应一个最优解；结点 D 、 J 是死结点，导致一个不可行解。

【题2】 $n = 4$, $W = 75$, $w = [10, 20, 30, 45]$, $v = [20, 20, 24, 36]$

问题的解空间如右图所示:



结点8产生问题的最优值76, 其解为 $(1, 1, 0, 1)$

考点4 着色问题（理解）

1、问题描述

若一个图最少需要 m 种颜色才能使图中任何一条边连接的2个顶点着有不同颜色，则称这个数 m 为该图的色数。求一个图的色数 m 的问题称为图的着色优化问题。

给定一个无向连通图 $G=(V,E)$ 和 m 种颜色，如果这个图不是 m 着色的，就回答“no”，如果这个图是 m 着色的，则要找出所有不同的着色方法。



扫码观看视频讲解更清晰

考点4 着色问题（理解）

2、问题分析

用邻接矩阵 a 表示无向连通图 $G=(V,E)$ ，若边 $(i,j)\in E$ ，则 $a[i][j]=1$ ，否则 $a[i][j]=0$ 。整数 $1, 2, \dots, m$ 表示 m 种不同的颜色。数组 $(x_1, x_2, \dots, x_n)$ 是问题的解向量，其中 x_i 表示顶点 i 所着的颜色。问题的解空间可以表示为高度为 $n+1$ 的完全 m 叉树。

在解空间树中每个内结点(叶子以外的结点)都有 m 个儿子，每个儿子对应的 x_i 都是 m 个可能的着色之一。解空间树中第 $n+1$ 层结点都是叶子结点。图的 m 着色问题解空间树中叶子结点个数是 m^n ，解空间树中每个内结点的孩子数是 m 。

考点4 着色问题（理解）

2、问题分析

现以顶点个数 $n=4$ ，颜色个数 $m=3$ 为例进行分析：

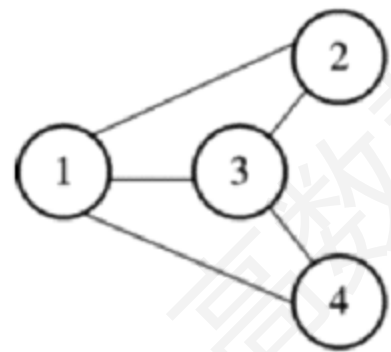


图1 4顶点的图

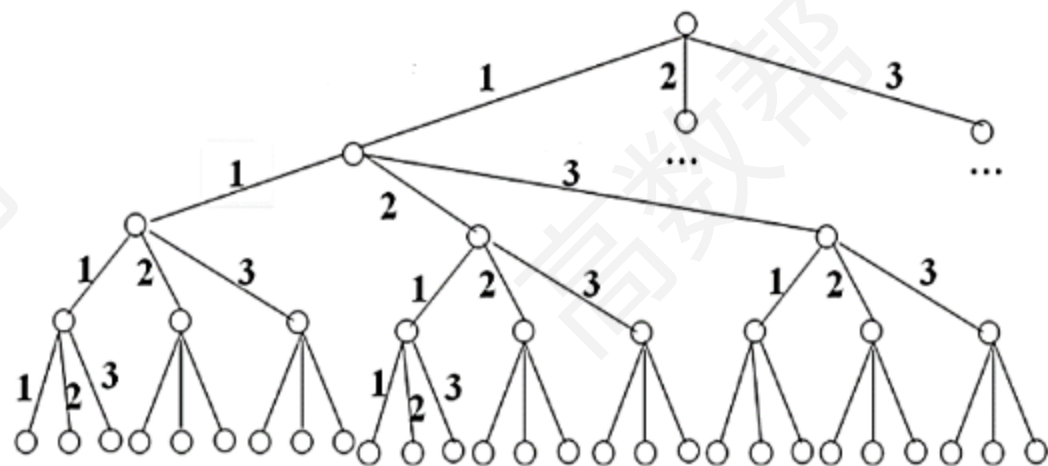


图2 3着色问题的解空间树

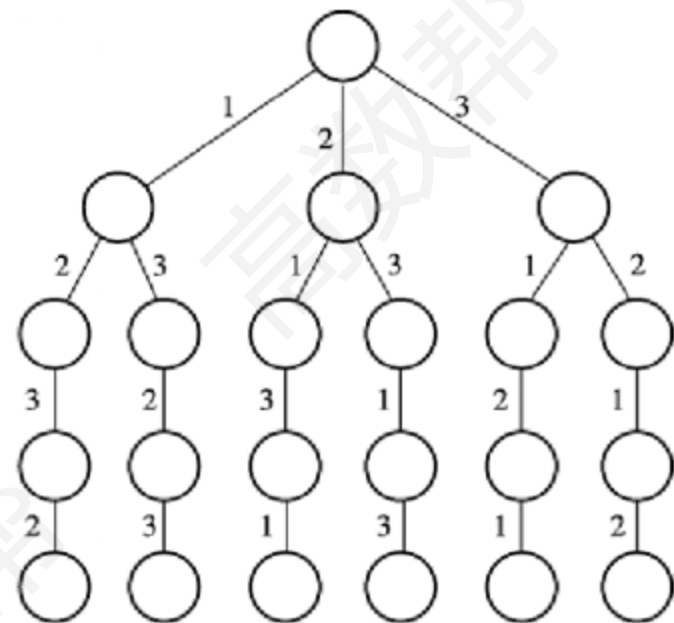


图3 所有可能的3着色

考点4 着色问题（理解）

3 算法复杂度分析

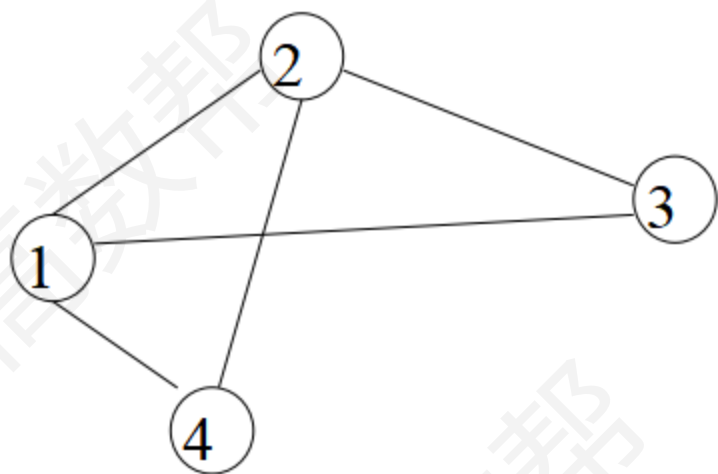
求解图的 m 着色问题的回溯算法的计算时间上界可以通过计算解空间树中结点个数来估计。图的 m 着色问题的解空间树中结点个数是 $\sum_{i=0}^n m^i$ 。

对于每个结点，在最坏情况下，确定每个结点的着色所需时间为 $O(mn)$ 。因此，该算法总的时间耗费是 $\sum_{i=0}^n m^i (mn) = O(nm^{n+1})$ 。



扫码观看视频讲解更清晰

【题3】 对于下图，写出图着色算法得出一种着色方案的过程。



解: $X[1] \leftarrow 1$, 返回 *true*

$X[2] \leftarrow 1$, 返回 *false*; $X[2] \leftarrow X[2] + 1 = 2$, 返回 *true*

$X[3] \leftarrow 1$, 返回 *false*; $X[3] \leftarrow X[3] + 1 = 2$, 返回 *false*; $X[3] \leftarrow X[3] + 1 = 3$, 返回 *true*

$X[4] \leftarrow 1$, 返回 *false*; $X[4] \leftarrow X[4] + 1 = 2$, 返回 *false*; $X[4] \leftarrow X[4] + 1 = 3$, 返回 *true*

找到一个解 (1, 2, 3, 3)

考点	重要程度	占分	题型
1. 分支限界法概述	★★★	2-5	填空、选择、简答
2. 单源最短路径问题	★★★	2-8	填空、算法设计
3. 0-1背包问题	★★★★★	2-4	填空、选择



扫码观看视频讲解更清晰

考点1 分支限界法概述（掌握）

分支限界法类似于回溯法，也是一种在问题的解空间树上搜索解的算法。

但分支限界法与回溯法有不同的求解目标。

回溯法的求解目标是找出解空间树中满足约束条件的所有解。

分支限界法的求解目标则是找出解空间树中使得某一目标函数值达到最小或最大时的解，即问题在某种意义下的最优解。

考点1 分支限界法概述（掌握）

由于求解目标不同，导致分支限界法与回溯法对解空间树的搜索方式有两点不同：

- （1）回溯法以深度优先的方式搜索解空间树；分支限界法以广度优先或以最小耗费（最大效益）优先的方式搜索解空间树。
- （2）回溯法一般只通过约束条件，而分支限界法不仅通过约束条件，而且通过目标函数的限界来减少无效搜索，提高求解效率。

考点1 分支限界法概述（掌握）

基本思想:

分支限界法常以广度优先或以最小耗费（最大效益）优先的方式搜索问题的解空间树。在分支限界法中，每一个活结点只有一次机会成为扩展结点。活结点一旦成为扩展结点，就一次性产生其所有儿子结点。在这些儿子结点中，导致不可行解或导致非最优解的儿子结点被舍弃，其余儿子结点被加入活结点表中。此后，从活结点表中取下一结点成为当前扩展结点，并重复上述结点扩展过程。这个过程一直持续到找到所需的解或活结点表为空时为止。

考点1 分支限界法概述（掌握）

从活结点表中选择下一扩展结点的不同方式导致不同的分支限界法，常见的分支限界法分为：

- ① 队列式分支限界法：将活结点表组织成一个队列，并按照队列先进先出（*FIFO*）原则选取下一个结点为扩展结点；
- ② 优先队列式分支限界法：将活结点表组织成一个优先队列（用堆实现），并选取优先级最高的活结点成为当前扩展结点。

考点2 单源最短路径问题（掌握）

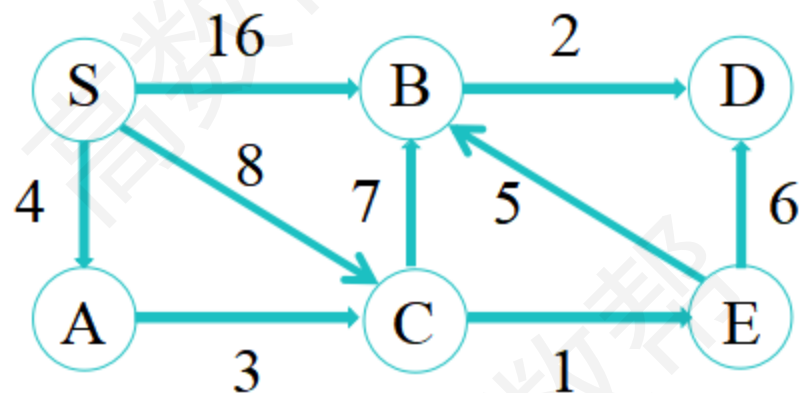
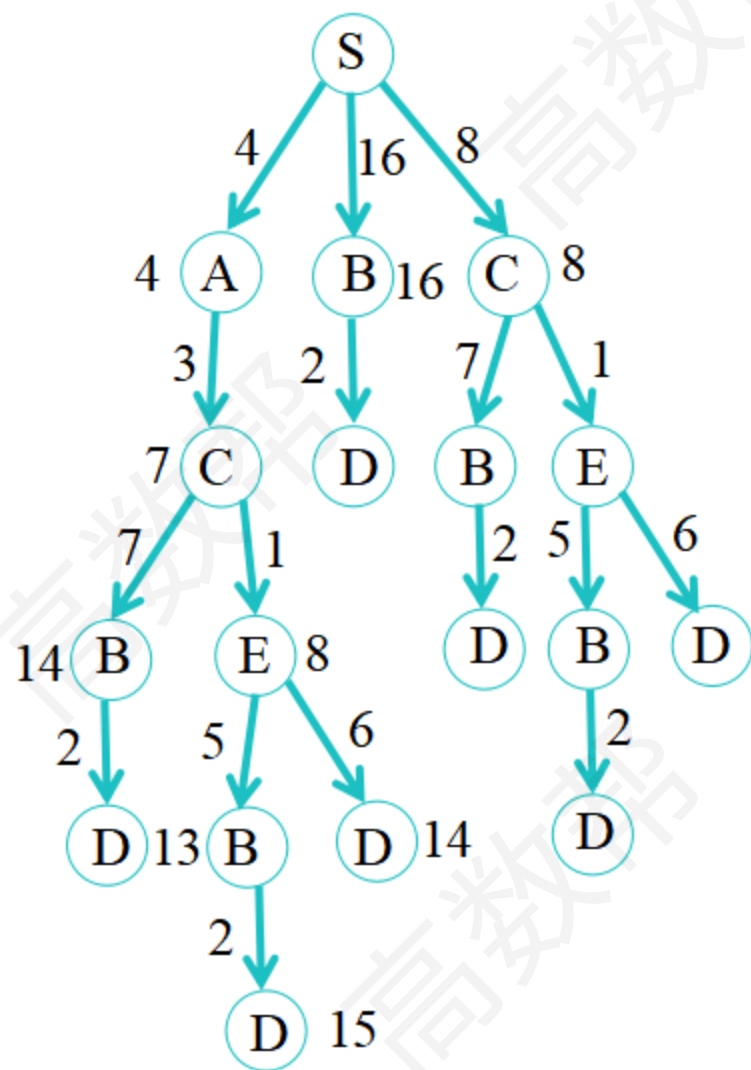
给定带权有向图 $G = (V, E)$,其中每条边的权是非负实数, 另外, 还给定 V 中的一个顶点, 称为源。目标是计算从源到所有其它各顶点的最短路径长度。这里路径长度是指路径上各边权值之和。



扫码观看视频讲解更清晰

【题1】 给定带权有向图，如下图所示，求源点 S 到顶点 D 的最短路径。

解： 该问题解空间树如图所示：



【题1】 给定带权有向图，如下图所示，求源点 S 到顶点 D 的最短路径。

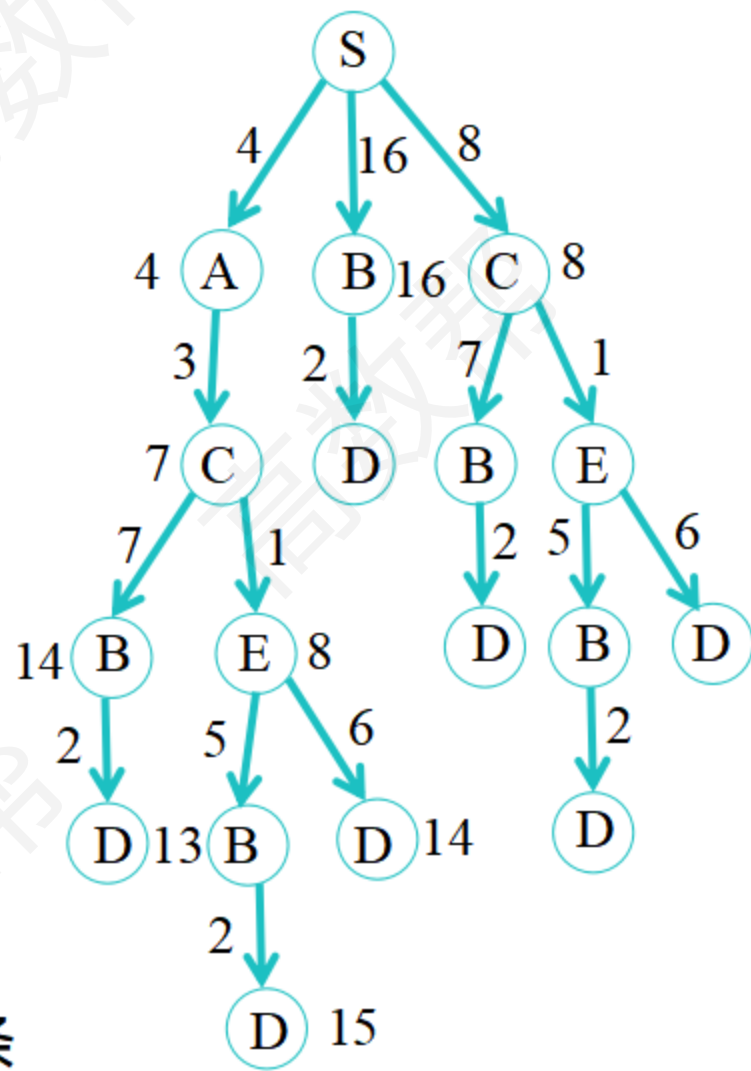
用优先队列式分支限界法求解

问题解的搜索过程为：

$Q = \{ \}$ ，初始化队列为空，放进去根节点 S ；

$Q = \{S\}$ ，结点 S 出队成为扩展结点生成结点 A 、 B 和 C ，
结点 A 、 B 和 C 可行且当前路长分别为4、16、8，可加入
队列，结点 A 成为下一个扩展结点；

$Q = \{A, B, C\}$ ，结点 A 出队成为扩展结点生成结 C ，结点 C 可
行且当前路长为7，比 $S \rightarrow C$ 这条路径上路长更小，因此可
加入队列，结点 C 成为下一个扩展结点，此外， $S \rightarrow C$ 这条
路径上结点 C 成为死结点；

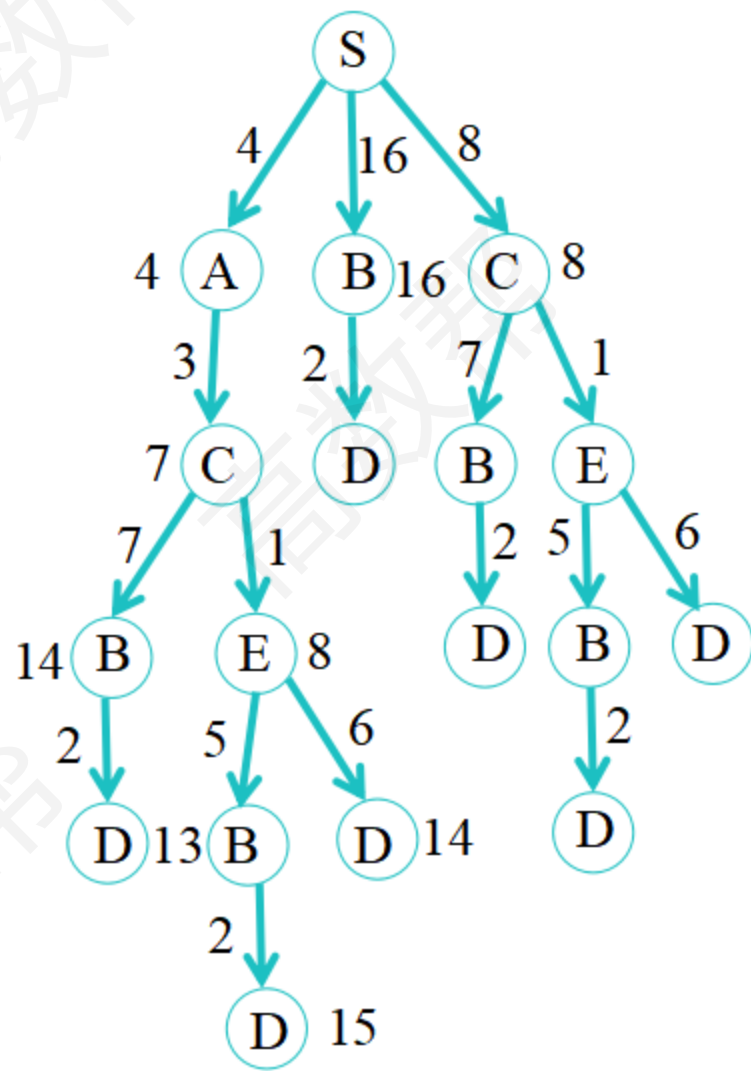


【题1】 给定带权有向图，如下图所示，求源点 S 到顶点 D 的最短路径。

$Q = \{D\}$ ，此时结点 D 是叶子结点，不能再扩展，
该节点当前路长为14；

$Q = \{ \}$ ，队列 Q 为空搜索结束。

由此就可得到 S 到 D 的一条最短路径， $S \rightarrow A \rightarrow C \rightarrow E \rightarrow D$ ，
最短路径长度为14。



考点3 0-1背包问题（理解）

一、问题描述

某商店有 n 个物品，第 i 个物品价值为 v_i ，重量（或权重）为 w_i ，背包的容量为 W 。每种物品只能选择完全装入或不装入背包，一个物品至多装入一次，因此该问题被称为0-1背包问题。目标是如何选择装入背包的物品，使装入背包的物品总价值最大。

$$\text{目标函数 } \max \sum_{1 \leq i \leq n} v_i x_i$$

$$\text{约束条件 } \sum_{1 \leq i \leq n} w_i x_i \leq W, x_i \in \{0, 1\}$$

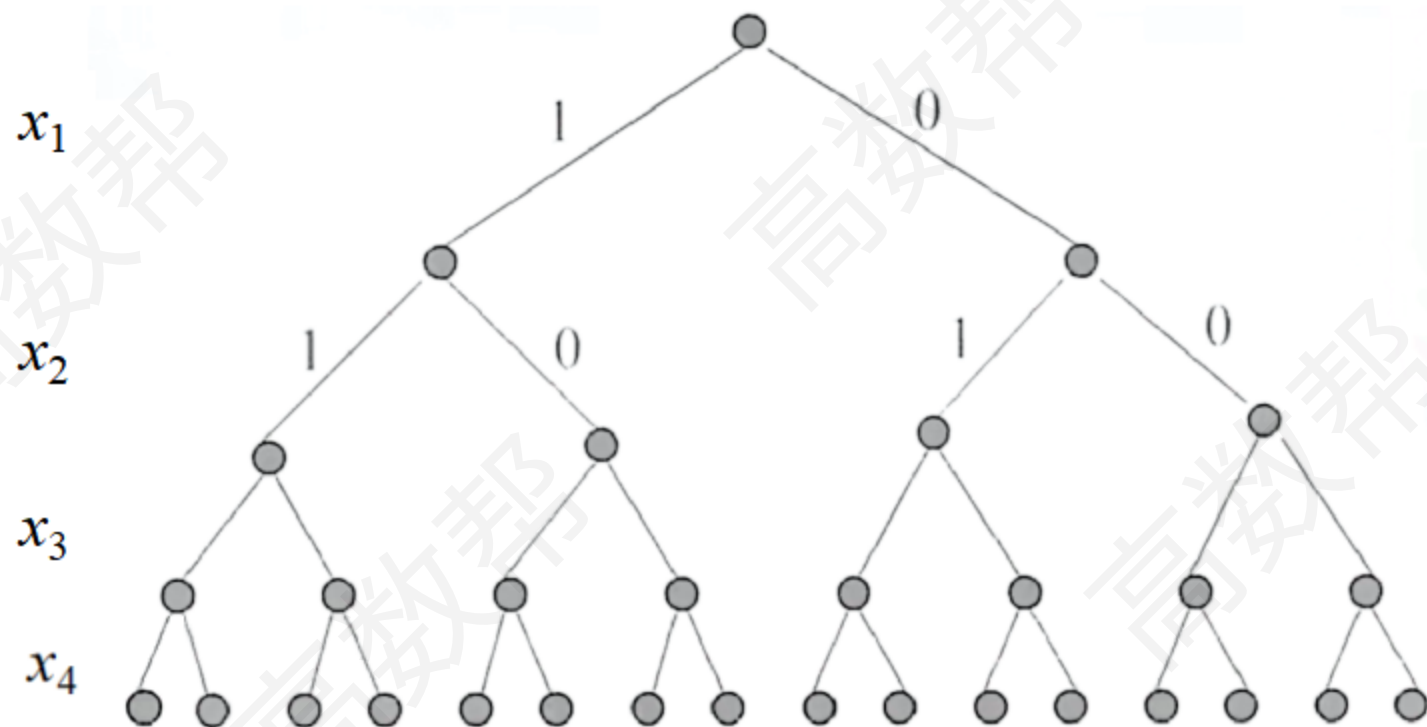


扫码观看视频讲解更清晰

考点3 0-1背包问题（理解）

二、分支限界法求解0-1背包问题

- 1、解空间：解用向量 $(x_1, x_2, \dots, x_n)$ 表示, $x_i \in \{0, 1\}$, $x_i = 1$ 表示第 i 个物品放入背包, $x_i = 0$ 表示第 i 个物品不放入背包。 $n = 4$ 时的解空间树如下图所示：



考点3 0-1背包问题（理解）

2、约束函数

用 $cw(i)$ 表示到达第 i 层时扩展结点的背包重量，即 $cw(i) = \sum_{j=1}^i w_j x_j$

约束函数为 $C(i) = cw(i) + w_{i+1}$ ，如果 $C(i) > W$ ，那么停止搜索该节点以下的层，否则继续搜索。

3、限界函数

对在第 i 层的扩展结点 N ，该结点 N 的价值为： $B(i) = cv(i) + r(i)$

($cv(i)$ 表示已确定放入的物品产生的价值， $r(i)$ 表示剩余可加入价值)

如果 $B(i) \leq bestv$ ，那么停止搜索以结点 N 为根结点的子树，结点 N 成为死结点，否则继续。

考点3 0-1背包问题（理解）

3、限界函数

$r(i)$ 的计算如下:

(1) 首先, 按照 $value/weight$ 的比值提前进行降序排序, 即

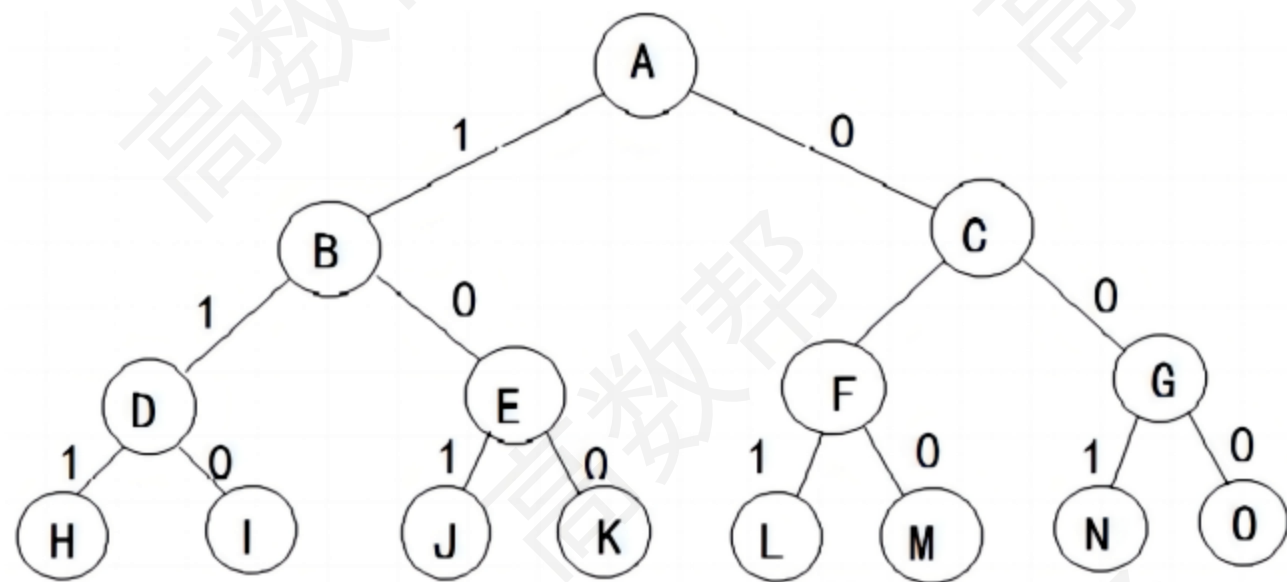
$$\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \dots \geq \frac{v_n}{w_n}$$

在第 i 层时, 背包还能装的容量为 $W - cw(i)$, 由贪心策略把剩下的物品放入背包中直到物品 k 不能再放进去为止, 得到:

$$r(i) = \sum_{j=i+1}^{k-1} v_j + \left(W - cw(i) - \sum_{j=i+1}^{k-1} w_j \right) \left(\frac{v_k}{w_k} \right)$$

【题2】 $n=3$, $W=30$, $w=[20,15,15]$, $v=[40,25,25]$

问题的解空间如右图所示:



扫码观看视频讲解更清晰

【题2】 $n = 3$, $W = 30$, $w = [20, 15, 15]$, $v = [40, 25, 25]$

(1) 用队列式分支限界法求解

问题解的搜索过程为：

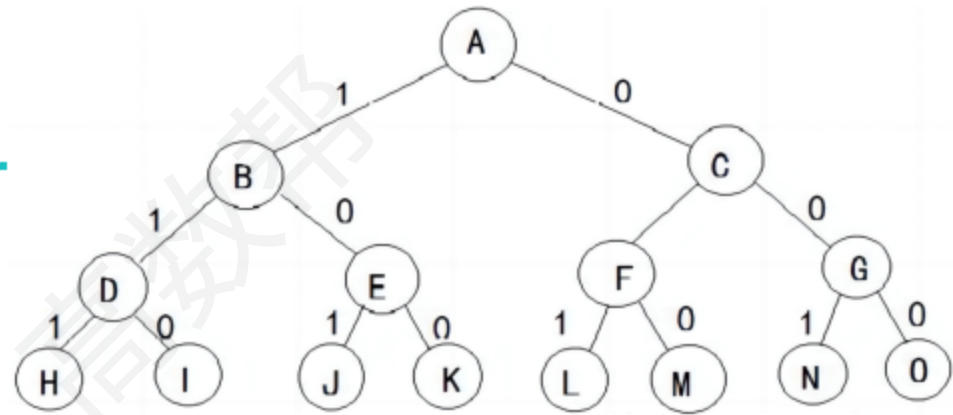
$Q = \{ \}$, 初始化队列 Q 为空, 从根节点开始, 把根节点 A 放入队列 Q ;

$Q = \{A\}$, 结点 A 出队成为扩展结点生成结点 B 和 C , 结点 B 和 C 可行, 加入队列;

$Q = \{B, C\}$, 结点 B 出队成为扩展结点生成结点 D 和 E , 结点 D 不可行, 超出背包容量, 成为死结点, 结点 E 可行, 加入队列;

$Q = \{C, E\}$, 结点 C 出队成为扩展结点生成结点 F 和 G , 结点 F 和 G 可行, 加入队列;

$Q = \{E, F, G\}$, 结点 E 出队成为扩展结点生成结点 J 和 K , 结点 J 不可行, 超出背包容量, 成为死结点, 结点 K 是可行的叶子节点, 产生问题的一个可行解, 它的价值为40;

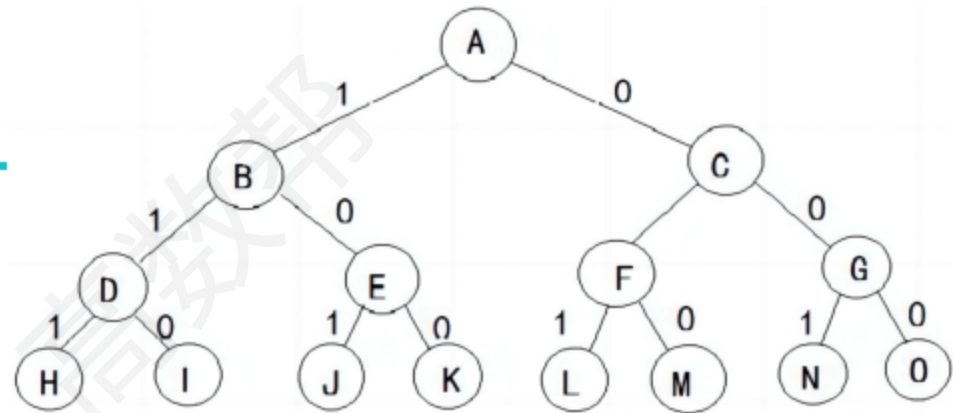


【题2】 $n = 3$, $W = 30$, $w = [20, 15, 15]$, $v = [40, 25, 25]$

$Q = \{F, G\}$, 结点 F 出列成为扩展结点生成结点 L 和 M , 结点 L 是可行的叶子节点, 产生问题的一个可行解, 它的价值为50, 结点 M 是可行的叶子节点, 产生问题的一个可行解, 它的价值为25;

$Q = \{G\}$, 结点 G 出列成为扩展结点生成结点 N 和 O , 结点 N 是可行的叶子节点, 产生问题的一个可行解, 它的价值为25, 结点 O 是可行的叶子节点, 产生问题的一个可行解, 它的价值为0;

$Q = \{ \}$, 搜索结束。在产生的所有可行解中, 结点 L 产生问题的最大价值, 对应的最优解为 $(0, 1, 1)$ 。



【题2】 $n = 3$, $W = 30$, $w = [20, 15, 15]$, $v = [40, 25, 25]$

(2) 用优先队列式分支限界法求解

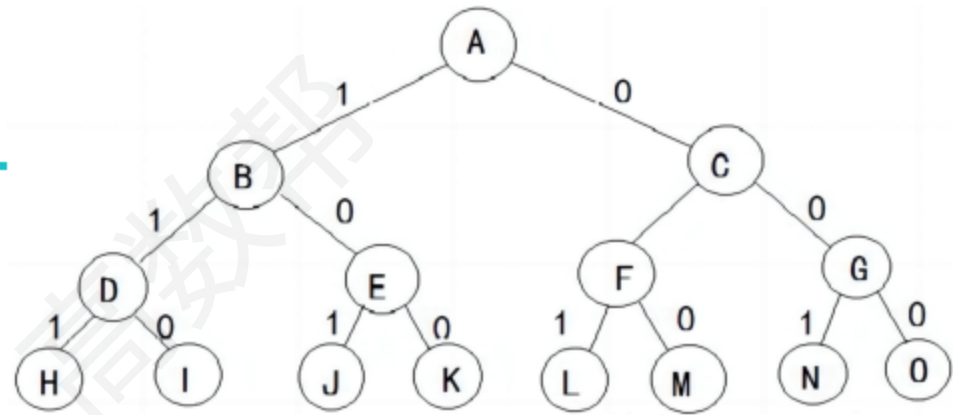
问题解的搜索过程为：

基于最大堆的优先队列 Q ，来维持待扩展结点的活结点表。

$Q = \{ \}$ ，初始化优先队列 Q 为空，从根节点开始，把根节点 A 放入优先队列 Q ；

$Q = \{A\}$ ，结点 A 出列成为扩展结点生成结点 B 和 C ，结点 B 的当前背包重量为20，价值上界为56.6，结点 C 的当前背包重量为20，价值上界为50，这两个结点都是可行结点，加入优先队列 Q 中；

$Q = \{B, C\}$ ，结点 B 出列成为扩展结点生成结点 D 和 E ，结点 D 的当前背包重量为35，超出实际背包容量，是不可行结点，结点 E 的当前背包重量为20，价值上界为56.6，是可行结点，加入优先队列 Q 中；



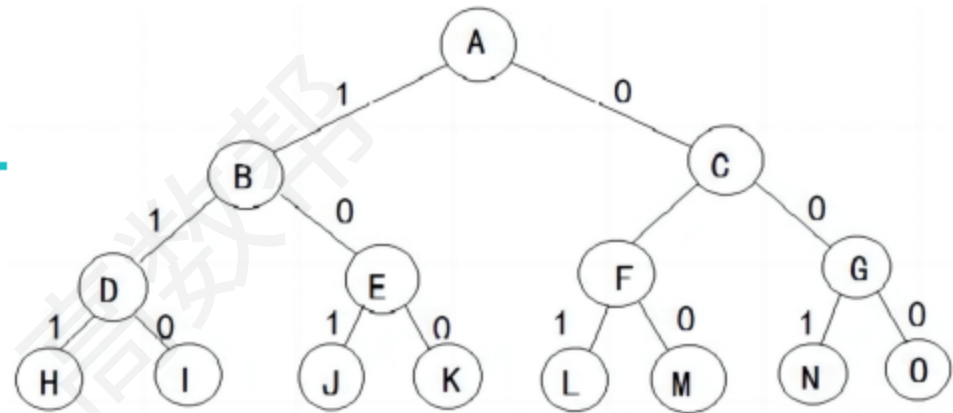
【题2】 $n=3$, $W=30$, $w=[20,15,15]$, $v=[40,25,25]$

$Q=\{E,C\}$, 结点 E 出列成为扩展结点生成结点 J 和 K , 结点 J 的当前背包重量为35, 超出实际背包容量, 是不可行结点, 结点 K 是可行的叶子节点, 产生问题的一个可行解, 它的总价值为40;

$Q=\{C\}$, 结点 C 出列成为扩展结点生成结点 F 和 G , 结点 F 的当前背包重量为15, 价值上界为50, 是可行结点, 加入优先队列 Q 中, 结点 G 的当前背包重量为0, 价值上界为25, 是不可行结点;

$Q=\{F\}$, 结点 F 出列成为扩展结点生成结点 L 和 M , 结点 L 是可行的叶子节点, 产生问题的一个可行解, 它的总价值为50, 结点 M 是不可行的叶子节点, 总价值为25小于50;

$Q=\{\}$, 搜索结束。问题的最优值为50, 对应的最优解为 $(0, 1, 1)$ 。





考点	重要程度	占分	题型
1. P 和 NP 问题	★★	2-5	填空、判断、简答



扫码观看视频讲解更清晰

考点1 P 和 NP 问题（理解）

1、 P (Polynomial, 多项式)类问题

P 类问题是指可以在多项式时间内求解的问题。

例如：排序问题、矩阵链乘问题、最小生成树问题等。

考点1 P 和 NP 问题（理解）

2、 NP (*Non-deterministic Polynomial*, 非确定性多项式)类问题

NP 类问题是指一个复杂问题不能确定是否能够在多项式时间内找到解，但是可以在多项式时间内验证一个解是否正确。例如：旅行售货商问题、团问题等。

旅行商问题(TSP): 假设有一个旅行商人要拜访 n 个城市，他必须选择所要走的路径，路径的限制是每个城市只能拜访一次，而且最后要回到原来出发的城市。目标是如何选择行进路线，以使总的行程最短。

这个问题的解空间是一棵排列树，排列树叶子节点有 $(n-1)!$ 个，任何算法遍历排列树所需时间复杂度为 $\Omega((n-1)!)$ ，这已经不是多项式时间的算法了。应该怎么办呢？猜想+验证。

考点1 P 和 NP 问题（理解）

3、 NPC (NP complete) 问题

归约：如果 A 问题可以在多项式时间内转化为 B 问题，就是说 A 可归约为 B 。

NPC 问题是指存在这样一个 NP 问题，所有的 NP 问题都可以归约成它。

换句话说，只要解决了这个问题，那么所有的 NP 问题都解决了。

其定义要满足2个条件：

- (1) 它是一个 NP 问题；
- (2) 所有 NP 问题都能规约到它。



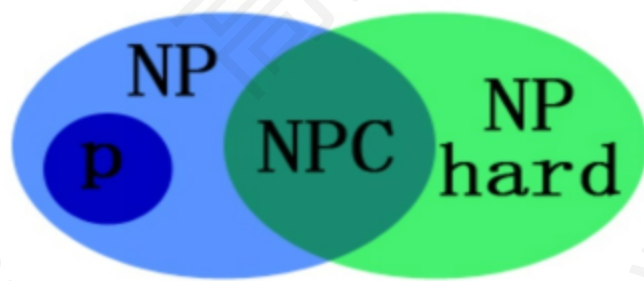
扫码观看视频讲解更清晰

考点1 P 和 NP 问题（理解）

4、 NP 难问题（ NP -hard问题）

NP -Hard问题是这样一种问题，它满足 NPC 问题定义的第二条但不一定要满足第一条即所有的 NP 问题都能归约到它，但是它不一定是一个 NP 问题。

以上四个问题之间的关系如下图所示：



扫码观看视频讲解更清晰